ALGORITHMS · PRINTABLE WORKBOOK

The Big-O Building

The definition of Big-O, learned on paper: a walkthrough, six hands-on activities, 45 questions in five tiers, and a full answer key that explains every wrong option too.

How to use this packet

1. Work through it in order. Each part starts with **Learn** pages, then **Try it** activities, then that tier's questions.
2. Use a pencil. Every question has space to mark your answer and, for multiple choice, a line to say why you ruled out the option you almost picked.
3. Each question names the **skill** it checks. Every wrong option is a real misconception somebody holds.
4. The **answer key** starts on its own page at the back, so you can tear it off. It explains *every* option, not only the right one. Read the explanations for the options you didn't pick too; that's where most of the learning is.
5. Some questions mention "the page" or "the walkthrough": that's the interactive site these come from. Part 1 here covers the same ground, including the building picture.

Part 1	The definition, on one example	9 questions
Part 2	Functions and code you have not seen	9 questions
Part 3	The work a problem set asks for	9 questions
Part 4	The whole family: O, Ω, Θ, o, ω	9 questions
Part 5	What any of this is for	9 questions
Summary	One page to keep	
Key	Answers to the activities and every question	

PART 1 · LEARN

What Big-O actually claims

Big-O answers one question: **as the input gets bigger, how fast does the amount of work grow?** The answer always comes as a *ceiling*.

The letters

- **n** is the size of the input, say how many items are in a list. It counts things, so it is a whole number.
- **T(n)** is the number of steps the code takes on an input of size n. The running example in this packet is $T(n) = 14n^2 + 4n + 6$.
- The pieces added together are its **terms**. The biggest, $14n^2$, is the **leading term**; $4n$ and 6 are the **lower order terms**.

$$T(n) = O(f(n))$$

means: **there is** a fixed number c > 0 (the constant) **and** a starting point n_0 such that

$$T(n) \le c \cdot f(n) \quad \text{for every } n \ge n_0$$

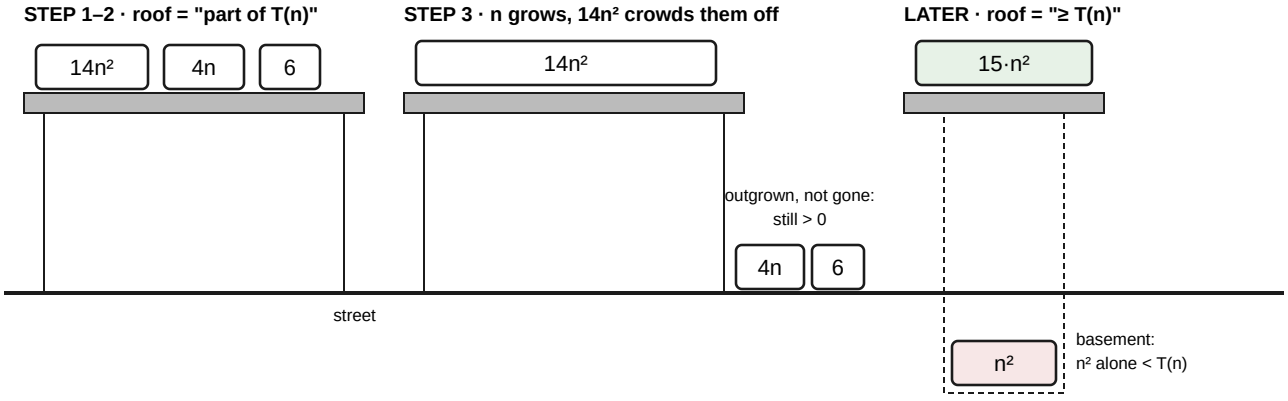
From n_0 on, c·f(n) never drops below T(n). That is the whole promise.

Because it is only a ceiling, it says **nothing** about how many seconds the code takes, **nothing** about small inputs, and **nothing** about T(n) being anywhere near the ceiling. $T(n) = 6$ is $O(n^2)$ too.

> **One convention, fixed before any arithmetic.** This packet writes $n \ge n_0$, the way Kleinberg & Tardos and CLRS do, so n_0 is the *first* n the promise covers. Some courses write $n > n_0$, making n_0 the *last* n allowed to fail. Same claim; every "smallest n_0" answer shifts by one. Check which one a problem set means.

The picture: a building

The interactive page tells this as a building, and several questions refer to it. T(n) starts on the roof with all its terms standing side by side. As n grows, $14n^2$ swells and shoulders the small terms off the edge onto the street. Later, a candidate ceiling c·f(n) is tested against the roof: on it if it is at or above T(n), sent to the basement if it is below.



> **The roof changes meaning once.** At first "on the roof" means *is a term of T(n)* (membership, not size, which is why 6 stands level with $14n^2$). Later it means *is at or above T(n)*. Under the first reading n^2 belongs on the roof; under the second it goes to the basement. Keep the two questions apart: *what is T(n) made of?* versus *what sits above T(n)?*

PART 1 · TRY IT

Find the constant and the threshold yourself

ACTIVITY A · FILL IN THE TABLE

$T(n) = 14n^2 + 4n + 6$. Fill every empty cell, then answer underneath. (The first row is done for you.)

n	T(n)	$14n^2$	$15n^2$	$15n^2 \geq T(n)$?	n^2	$4n + 6$	$n^2 \geq 4n + 6$?
1	24	14	15	no	1	10	no
2							
3							
4							
5							
6							
7							
8							

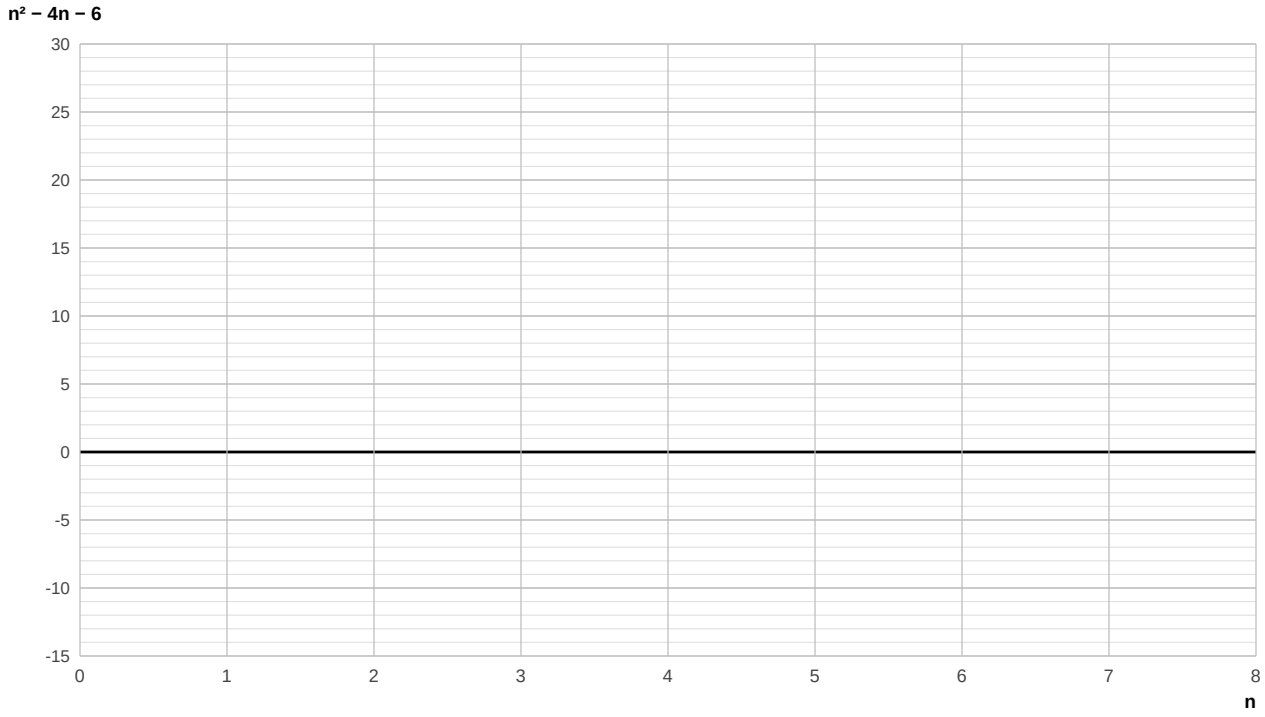
1. Is $14n^2 \geq T(n)$ in *any* row? Cancel $14n^2$ from both sides of $14n^2 \geq 14n^2 + 4n + 6$. What is left, and why can it never be true?

2. What is the first n where $15n^2 \geq T(n)$? Compare the two "?" columns. Why do they flip at the same row?

3. At n = 11, 4n + 6 is 50 and T(11) is 1744, under 3 percent. Does that make 4n + 6 "negligible" for the inequality?

ACTIVITY B · PLOT THE GAP

The distance between the ceiling and T(n) is $15n^2 - T(n) = n^2 - 4n - 6$. Plot it for n = 0 … 8 and join the dots. Where it is **below** zero, the ceiling is short. Shade that stretch. Where does the curve cross zero (roughly)? Why is n_0 the next *whole* number and not the crossing itself?



Why the constant is 15, not 14

The small terms are outgrown, not killed

4n + 6 does not shrink as n grows; it grows too. What shrinks is its *share*, `(4n + 6) / T(n)`. **A small share is not a small number.** 14n² on its own falls short of T(n) by exactly 4n + 6, at every n, forever. So c = 14 never works, not for any n_0.

Raising c pays for what you stopped naming

Raising the constant by one is worth exactly one whole n², because `15n² − 14n² = n²`. The question becomes: is one spare n² enough to cover 4n + 6? That is `n² ≥ 4n + 6`, which first holds at n = 6 (at n = 5: 25 vs 26, still short by one; at n = 6: 36 vs 30). The two curves cross at n ≈ 5.16, and n counts things, so the first whole n past the crossing is the answer: **$n_0 = 6$**.

> The moment the *shape* n² outgrows the terms you dropped is exactly the moment c = 15 starts working. `15n² ≥ T(n)` and `n² ≥ 4n + 6` are the same inequality.

What multiplying by c does

c·f(n) **stretches** the curve vertically. It never bends it. That is the whole mechanism: n² is too small by itself (it rides the elevator to the basement), and 15·n² is the same shape lifted until it clears T(n) from n_0 on. The constant is the elevator ticket. It can never turn one shape into another: `c·n² ≥ n³` rearranges to `c ≥ n`, which fails the moment n passes c.

The word **asymptotic** names the "from some point onward" part: tiny inputs may be messy; only the long run counts.

The ceiling does not have to be low

ceiling	c	holds from	check
`15n²`	15	n = 6	540 ≥ 534 at n = 6
`100n²`	100	n = 1	100 ≥ 24 at n = 1
`n³`	1	n = 15	n = 14: 2744 < 2806; n = 15: 3375 ≥ 3216

All three are true. So T(n) = O(n²) *and* T(n) = O(n³) (and O(n⁴)…), each weaker than the last. n² is worth saying because it is the lowest ceiling that fits. That stronger claim, pinned from both sides, is **Θ** ("theta"): `14n² ≤ T(n) ≤ 15n²` for every n ≥ 6, so T(n) = Θ(n²). Big-O alone only ever promises the ceiling.

> Trap: n³ meets 14n² at exactly n = 14 (both 2744). That is *not* where n³ passes T(n); the bound is against T(n), not its leading term. n³ is still 62 short at n = 14.

ACTIVITY C · C AND N_0 TRADE AGAINST EACH OTHER

For each constant, find the smallest n_0 with $T(n) \le c \cdot n^2$ for every $n \ge n_0$. Hint: $T(n) \le c \cdot n^2$ is $(c - 14)n^2 - 4n - 6 \ge 0$.

c	14	15	16	17	18	50
smallest n_0						

What happens to n_0 as c goes up? Is any of these pairs a "better" proof that $T(n) = O(n^2)$?

PART 1 · QUESTIONS

Tier 1: The definition, on one example

1.1 *Skill: what a bound promises*

An algorithm's running time is $T(n) = O(n^2)$. Which of these follows?

- ◯ **A.** On an input of size 1000 it takes about 1,000,000 steps.
- ◯ **B.** It is slower than an O(n) algorithm on large inputs.
- ◯ **C.** It takes at least some fixed multiple of n^2 steps.
- ◯ **D.** From some n^0 on, its step count stays under a fixed multiple of n^2. Nothing more.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.2 *Skill: what multiplying by c does*

Multiplying f(n) by a larger constant does what to its curve?

- ◯ **A.** Shifts it up by c.
- ◯ **B.** Stretches it vertically about the n axis. The shape is untouched.
- ◯ **C.** Makes it grow faster, so a big enough c can make $c \cdot n^2$ overtake n^3.
- ◯ **D.** Moves it rightward, so the crossing happens later.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.3 *Skill: what height means in the drawing*

In the opening steps $14n^2$, $4n$ and 6 all stand level on the roof, even though $14n^2$ dwarfs 6. Later n^2 is sent to the basement for being too small. What changed?

- ◯ **A.** Nothing changed. The drawing is inconsistent.
- ◯ **B.** The roof always meant "at or above T(n)", and putting 6 up there was a slip.
- ◯ **C.** n grew, so 6 stopped being a term of T(n).
- ◯ **D.** Height first meant "is a term of T(n)". Once the chips come back up as $c \cdot f(n)$ it means "is at or above T(n)".

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.4 *Skill: why the small terms can be dropped*

```
T(n) = 14n² + 4n + 6
```

Why is it legitimate to leave $4n + 6$ out of the name of the bound?

- ◯ **A.** Because $4n + 6$ becomes negligibly small.
- ◯ **B.** Because Big-O ignores lower order terms by definition.
- ◯ **C.** Because a constant can be raised to cover them: one more unit of c buys a whole n^2, which outgrows $4n + 6$ from $n = 6$.
- ◯ **D.** Because $4n + 6$ is smaller than $14n^2$ at every n.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.5 *Skill: what the constant has to cover*

```
T(n) = 14n² + 4n + 6, f(n) = n²
```

A reader says: $c = 14$ is fine, you just have to pick n_0 big enough, because $4n + 6$ is negligible next to $14n^2$. Where does that go wrong?

- ◯ **A.** $14n^2 \geq T(n)$ cancels down to $0 \geq 4n + 6$, and $4n + 6$ is positive at every n. No n_0 can rescue it.
- ◯ **B.** Nowhere. The share $(4n + 6) / T(n)$ really does go to zero, so past some n_0 the inequality holds.
- ◯ **C.** It fails only for small n. Setting $n_0 = 1000$ fixes it.
- ◯ **D.** It fails because c must always be strictly greater than the leading coefficient.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.6 *Skill: how many n_0 there are*

> $T(n) = 14n^2 + 4n + 6$, $f(n) = n^2$, $c = 15$

Tick every n_0 that makes $T(n) \le 15n^2$ true for all $n \ge n_0$.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $n^0 = 3$
- ☐ **B.** $n^0 = 4$
- ☐ **C.** $n^0 = 5$
- ☐ **D.** $n^0 = 6$
- ☐ **E.** $n^0 = 10$
- ☐ **F.** $n^0 = 100$

1.7 *Skill: what a bigger constant buys*

> $T(n) = 14n^2 + 4n + 6$, $f(n) = n^2$, $c = 18$

With c = 15 the smallest n_0 is 6. Raise the constant to c = 18. What is the smallest n_0 now?

n_0 = ☐

Working:

1.8 *Skill: which crossings actually happen*

Put these in order of eventual size, smallest first. Eventual means: once n is large enough that the order stops changing.

A $14n^2$	**B** n^3	**C** $4n + 6$	**D** 6	**E** n^2

1 ☐ < 2 ☐ < 3 ☐ < 4 ☐ < 5 ☐

Write the letters in the boxes, slowest-growing / smallest first. For each neighbouring pair, can you say where they cross?

1.9 *Skill: how many bounds are true at once*

> $T(n) = 14n^2 + 4n + 6$

Tick every true statement.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $T(n) = O(n^2)$
- ☐ **B.** $T(n) = O(n^3)$
- ☐ **C.** $T(n) = O(n)$
- ☐ **D.** $T(n) = \Theta(n^2)$
- ☐ **E.** $T(n) = \Theta(n^3)$
- ☐ **F.** $T(n) = O(n^2 \log n)$

PART 2 · LEARN

Reading growth off code, and the ladder

What a loop does to its counter decides the shape

Pattern	Count	Bound
counter goes down (or up) by 1 each pass	n passes	O(n)
counter is **divided** (halved) each pass	about $\log_2 n$ passes (n = 1000 → 9)	O(log n)
loop inside a loop, both over n	n · n	$O(n^2)$
inner loop runs i times (i = 0 … n−1)	0 + 1 + … + (n−1) = n(n−1)/2	still $\Theta(n^2)$
stage A, **then** stage B (one after the other)	cost(A) + cost(B)	the larger one
stage B **inside** stage A (nested)	cost(A) × cost(B)	the product

Halving the work does not change the class: the ½ is a constant factor and disappears into c.

The ladder everything gets read against

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

The gaps are not the same size. n and n log n are near neighbours (the log is 20 at a million); n^2 and 2^n are worlds apart. Every exponential eventually beats every polynomial, even n^{1000} (the crossing is near n = 1000, and Big-O does not care how late it is).

Rules of thumb, and where they break

- **Constant factors are forgiven**: $2^{n+1} = 2 \cdot 2^n = O(2^n)$; $\log(n^2) = 2 \log n = O(\log n)$; the base of a log never matters ($\log_2 n = \log n / \log 2$).
- **Constants inside an exponent are not**: $2^{2n} = 4^n$, and $4^n/2^n = 2^n$ passes every c. $(\log n)^2$ is not O(log n).
- **f = O(g) runs one way.** n = O(n log n) but not the reverse. When it runs both ways the pair is Θ of each other.
- **O(1)** means the cost stops depending on n. Not "one step", not "always the same", not "fast": a routine that always takes 40,000 steps is O(1).
- **O is not "worst case".** Worst, best and average case are three different functions of n; O, Ω, Θ are three ways to bound a function. Quicksort's worst case is $O(n^2)$ and its best case is O(n log n); both are Big-O statements.

ACTIVITY D · COUNT IT

```
k = n
while k > 1:
    k = k // 2
    step()
```

n	1	2	8	10	100	1000
step() calls						

```
for i in range(n):
    for j in range(i):
        step()
```

n	1	2	3	4	5	10
step() calls						
n(n−1)/2						

PART 2 · QUESTIONS

Tier 2: Functions and code you have not seen

2.1 *Skill: what O(1) claims*

A lookup is O(1). Which is true?

- ◯ **A.** Its cost stops depending on n. The fixed cost can still be large, and it can still vary from call to call.
- ◯ **B.** It takes one step.
- ◯ **C.** It is faster than any O(log n) operation.
- ◯ **D.** It always takes the same amount of time.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.2 *Skill: what Big-O is a statement about*

True or false: Big-O means the worst case.

- ◯ **A.** True. O is the worst case and Ω is the best case.
- ◯ **B.** False. Big-O bounds a function, and which function you chose to bound is a separate decision.
- ◯ **C.** True, because Big-O takes the maximum over all inputs of that size.
- ◯ **D.** False, because Big-O is about the average case.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.3 *Skill: dividing against decrementing*

```
k = n
while k > 1:
    k = k // 2      # halve k, rounding down
    step()
```

What is the tightest bound, the lowest ceiling that fits, on the number of step() calls?

- ◯ **A.** O(n)
- ◯ **B.** O($\log^2$ n), and the base matters here.
- ◯ **C.** O(log n)
- ◯ **D.** O(1), since the loop body is a single operation.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.4 *Skill: counting a loop whose bound moves*

```
for i in range(n):        # i takes the values 0, 1, ..., n - 1
    for j in range(i):    # i passes this time round, not n
        step()
```

How many times does step() run, and what is the tightest correct bound?

- ◯ **A.** n^2 calls, so O(n^2).
- ◯ **B.** About $n^2 / 2$ calls, so O(n^2), and that bound is tight.
- ◯ **C.** About $n^2 / 2$ calls, so O(n^2), but not tight, because the inner loop only runs half the time.
- ◯ **D.** O(n log n), since the inner loop is shorter each time.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.5 *Skill: adding two bounds against nesting them*

A routine sorts in O(n log n), then makes a separate O(n^2) pass over the result. What is the tightest bound on the whole routine, in the simplest form that is still correct?

- ◯ **A.** O(n^2 log n)
- ◯ **B.** O(n log n + n^2)
- ◯ **C.** O(n^2)
- ◯ **D.** O(n log n), since sorting is the expensive stage.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.6 *Skill: the ladder everything gets read against*

Put these in order of growth, slowest growing first.

A n	**B** n log n	**C** √n	**D** n^2	**E** n!	**F** log n	**G** 2^n

[1] < [2] < [3] < [4] < [5] < [6] < [7]

Write the letters in the boxes, slowest-growing / smallest first. For each neighbouring pair, can you say where they cross?

2.7 *Skill: how large n_0 is allowed to be*

> $T(n) = 3n^2 + 100n + 5000$, $f(n) = n^2$, $c = 4$

The smallest whole c that works here is 4. With c = 4, what is the smallest n_0?

n_0 = [ ]

Working:

2.8 *Skill: which rewrites are legal*

Tick every true statement.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $2^{n+1} = O(2^n)$
- ☐ **B.** $2^{2n} = O(2^n)$
- ☐ **C.** $\log(n^2) = O(\log n)$
- ☐ **D.** $(\log n)^2 = O(\log n)$
- ☐ **E.** $n! = O(n^n)$
- ☐ **F.** $n^n = O(n!)$

2.9 *Skill: which way each bound runs*

Tick every pair where f(n) = O(g(n)).

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $f = n^2$, $g = n^2 + n$
- ☐ **B.** $f = n^2 + n$, $g = n^2$
- ☐ **C.** $f = n \log n$, $g = n$
- ☐ **D.** $f = n$, $g = n \log n$
- ☐ **E.** $f = 2^n$, $g = n^{100}$
- ☐ **F.** $f = n^{100}$, $g = 2^n$

Proving things from the definition

The quantifiers are the claim

∃ means "there is at least one", ∀ means "for every", ¬ means "it is not the case that".

$$f(n) = O(g(n)) \iff \exists c > 0 \;\; \exists n_0 \;\; \forall n \geq n_0 : f(n) \leq c \cdot g(n)$$

Read left to right: each thing is chosen knowing only what stands to its left. c and n_0 are fixed *once*, then every n from n_0 on must obey. Move ∃c inside ∀n and the constant may change with n, which makes the condition true of everything and so worthless. Change ∃c to ∀c and you get the much stronger **o** ("little o").

A proof of f = O(g) is a witness plus a check

Name a c and an n_0, then show the inequality holds for every $n \geq n_0$. Nothing asks for the smallest c. **Any pair that holds is a correct proof.** Rearranging algebra without naming the pair is not a proof.

Negating it

Walk left to right: every ∃ becomes ∀ and every ∀ becomes ∃, **nothing changes places**, and the relation at the end flips:

$$f \neq O(g) \iff \forall c > 0 \;\; \forall n_0 \;\; \exists n \geq n_0 : f(n) > c \cdot g(n)$$

In words: whatever c and n_0 somebody offers, there is an n at or past n_0 that breaks them. (Two ∀ next to each other may swap; order only matters across a change of quantifier.)

Proof by contradiction: the template

1. **Assume** f = O(g): there are *some* c > 0 and n_0 with $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. You do not get to choose them.
2. **Rearrange** keeping c as a letter (divide by something positive, keep the direction).
3. **Build the breaking n out of c** (and make sure it is $\geq n_0$, e.g. take the larger of the two). It must pass the ceiling strictly, not just reach it.
4. **Conclude**: this contradicts the assumed pair, whichever it was, so no pair exists.

> **The classic slip**: writing "n log n ≤ n" on line one. That silently sets c = 1, and the contradiction then rules out c = 1 and nothing else.

> **Another**: "f is $O(n^2)$ and g is $O(n^2)$, so f = O(g)". A shared ceiling relates each function to the ceiling, not to each other (n^2 and n are both $O(n^2)$; n^2 is not O(n)). Transitivity is f = O(g) and g = O(h) ⇒ f = O(h).

PART 3 · TRY IT

Where are the failures?

ACTIVITY E · THE TIER'S RUNNING EXAMPLE

$f(n) = n^2$ and $g(n) = (n - 1)(n - 2) + 1 = n^2 - 3n + 3$. Fill in the table and mark each $c \cdot g$ column ✓ where $c \cdot g(n) \geq f(n)$.

n	$f(n) = n^2$	$g(n)$	$1 \cdot g$	$2 \cdot g$	$3 \cdot g$	$4 \cdot g$
0						
1						
2						
3						
4						
5						
6						

1. For $c = 3$, which n fail? Is it the small ones? What is the smallest n_0?

2. For $c = 2$, what is the smallest n_0?

3. Show that $4g(n) - f(n)$ is a perfect square times a constant. What does that tell you about $c = 4$?

4. Why does $c = 1$ never work? (Cancel.)

Tier 3: The work a problem set asks for

3.1 *Skill: the order of the quantifiers*

> f and g are positive functions of n. ∃ means "there is at least one", ∀ means "for every"

Which of these is the definition of f(n) = O(g(n))?

- ◯ **A.** $\exists c > 0\ \exists n_0\ \forall n \geq n_0 : f(n) \leq c \cdot g(n)$
- ◯ **B.** $\forall c > 0\ \exists n_0\ \forall n \geq n_0 : f(n) \leq c \cdot g(n)$
- ◯ **C.** $\exists c > 0\ \forall n_0\ \exists n \geq n_0 : f(n) \leq c \cdot g(n)$
- ◯ **D.** $\exists n_0\ \forall n \geq n_0\ \exists c > 0 : f(n) \leq c \cdot g(n)$

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.2 *Skill: producing a witness rather than recognising one*

> $f(n) = n^2$, $g(n) = (n - 1)(n - 2) + 1 = n^2 - 3n + 3$

Prove f(n) = O(g(n)) straight from the definition: name a constant c and a threshold n_0 that make $f(n) \leq c \cdot g(n)$ hold for every $n \geq n_0$.

c = _____ **n_0 =** _____

Check your pair. Show that the inequality holds at every $n \geq n_0$ (a table of a few values is evidence, not proof: say why it keeps holding).

3.3 *Skill: which n decides the constant*

> $f(n) = n^2$, $g(n) = n^2 - 3n + 3$, $n_0 = 1$

Pin the threshold at $n_0 = 1$, so the inequality has to hold at every $n \geq 1$. What is the smallest c that works?

c = _____

Working:

3.4 *Skill: where the failures actually are*

> $f(n) = n^2$, $g(n) = n^2 - 3n + 3$, $c = 3$

With c = 3, what is the smallest n_0 that makes $f(n) \leq 3 \cdot g(n)$ hold for every $n \geq n_0$?

n_0 = _____

Working:

3.5 *Skill: whether an argument proves what it says it proves*

> $f(n) = n^2$, $g(n) = n^2 - 3n + 3$, $s(n) = -3n + 3$, so $g = f + s$

An answer reads: f is $O(n^2)$, and s is $O(n^2)$, so by additivity, the rule that a sum of two $O(n^2)$ functions is itself $O(n^2)$, g = f + s is $O(n^2)$. Therefore f is O(g). Where does it break?

- ◯ **A.** Both f and g being $O(n^2)$ does not make either one O of the other.
- ◯ **B.** Nothing is wrong. Both steps are standard.
- ◯ **C.** The additivity step is wrong: $O(n^2)$ plus $O(n^2)$ is $O(n^4)$.
- ◯ **D.** $s(n) = -3n + 3$ goes negative, so it has no Big-O bound at all.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.6 *Skill: how general a proof by contradiction is*

```
Claim: n log₂ n is not O(n).

"Assume it is. Then n log₂ n ≤ n for all n ≥ n₀.
 Dividing by n gives log₂ n ≤ 0 for all n ≥ n₀.
 But log₂ n ≤ 0 only at n = 1, so no such n₀ exists.
 Contradiction."
```

Every line of arithmetic here is right and the conclusion is true. What has the argument not established?

- ◯ **A.** It rules out c = 1 and nothing else. The assumption hands over some constant, not that one.
- ◯ **B.** Nothing. Taking c = 1 is without loss of generality, since a larger c can be absorbed into n^0.
- ◯ **C.** The division by n is not allowed, because n could be 0.
- ◯ **D.** The contradiction is with $\log^2 n \le 0$, which is a fact about logarithms rather than about T(n).

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.7 *Skill: both directions at once*

```
f(n) = n², g(n) = n² − 3n + 3
```

Tick every true statement.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** f(n) = O(g(n))
- ☐ **B.** g(n) = O(f(n))
- ☐ **C.** f(n) = Θ(g(n))
- ☐ **D.** f(n) − g(n) = O(1)
- ☐ **E.** g(n) = O(n)
- ☐ **F.** f(n) ≤ 3·g(n) fails at n = 2, so f(n) = O(g(n)) fails at n = 2

3.8 *Skill: negating a statement with four quantifiers in it*

> `f(n) = O(g(n)) is ∃c > 0 ∃n₀ ∀n ≥ n₀ : f(n) ≤ c·g(n). ¬ means "it is not the case that"`

Tick every correct negation. There is more than one.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $\forall c > 0\ \forall n^{0}\ \exists n \geq n^{0} : f(n) > c \cdot g(n)$
- ☐ **B.** $\forall c > 0\ \forall n^{0}\ \neg(\forall n \geq n^{0} : f(n) \leq c \cdot g(n))$
- ☐ **C.** $\forall n^{0}\ \forall c > 0\ \exists n \geq n^{0} : f(n) > c \cdot g(n)$
- ☐ **D.** $\exists c > 0\ \exists n^{0}\ \forall n \geq n^{0} : f(n) > c \cdot g(n)$
- ☐ **E.** $\forall c > 0\ \exists n^{0}\ \forall n \geq n^{0} : f(n) > c \cdot g(n)$
- ☐ **F.** $\exists c > 0\ \forall n^{0}\ \exists n \geq n^{0} : f(n) > c \cdot g(n)$

3.9 *Skill: writing the contradiction, not recognising it*

> $T(n) = n \log_2 n$

Build the proof by contradiction that T(n) is not O(n).

LINE 1 · THE ASSUMPTION

- ◯ **A.** Suppose T(n) = O(n). Then there are some c > 0 and some n^{0} with $n \log^{2} n \leq c \cdot n$ for every $n \geq n^{0}$.
- ◯ **B.** Suppose T(n) = O(n). Then $n \log^{2} n \leq n$ for every $n \geq n^{0}$.
- ◯ **C.** Suppose T(n) = O(n). Then for every c > 0 there is an n^{0} with $n \log^{2} n \leq c \cdot n$ for every $n \geq n^{0}$.

LINE 2 · THE REARRANGEMENT

- ◯ **A.** Divide both sides by n: $\log^{2} n \leq 0$ for every $n \geq n^{0}$.
- ◯ **B.** Subtract n from both sides: $n \log^{2} n - n \leq 0$, so $n(\log^{2} n - 1) \leq 0$.
- ◯ **C.** Divide both sides by n, positive at every n the claim covers: $\log^{2} n \leq c$ for every $n \geq n^{0}$.

LINE 3 · THE N THAT BREAKS IT

- ◯ **A.** Take n = 2^(c+1), or n^{0} if that is larger. Then $n \geq n^{0}$ and $\log^{2} n \geq c + 1$.
- ◯ **B.** Take n = 2^c. Then $\log^{2} n = c$.
- ◯ **C.** Take $n = n^{0} + 1$. Then $\log^{2} n > c$.

LINE 4 · THE CONCLUSION

- ◯ **A.** So c = 1 does not work. Since no constant works, $n \log^{2} n$ is not O(n).
- ◯ **B.** This n is at least n^{0} and has $\log^{2} n > c$, contradicting $\log^{2} n \leq c$. So no such c and n exist, and $n \log^{2} n$ is not O(n).
- ◯ **C.** So $\log^{2} n \leq c$ is false, and therefore $n \log^{2} n = \Omega(n \log n)$.

One line per step. The proof counts only if every line is right.

PART 4 · LEARN

The family: O, Ω, Θ, o, ω

Say	Reads like	Definition (from some n_0 on)	Meaning
f = O(g)	≤	$\exists c\ \exists n_0\ \forall n \geq n_0$: $f(n) \leq c \cdot g(n)$	ceiling
f = Ω(g)	≥	$\exists c\ \exists n_0\ \forall n \geq n_0$: $f(n) \geq c \cdot g(n)$	floor
f = Θ(g)	=	both: $c_1 \cdot g \leq f \leq c_2 \cdot g$	tight, same shape
f = o(g)	<	$\forall c\ \exists n_0\ \forall n \geq n_0$: $f(n) \leq c \cdot g(n)$	falls behind by every factor
f = ω(g)	>	$\forall c\ \exists n_0\ \forall n \geq n_0$: $f(n) \geq c \cdot g(n)$	pulls away by every factor

- **Transpose symmetry:** f = O(g) ⇔ g = Ω(f), and f = o(g) ⇔ g = ω(f). Θ is its own transpose, which is why it reads like equality.
- **"At least O(n^2)" is empty.** O is already a ceiling; putting a floor word in front rules nothing out. Say Ω(n^2).
- **Where the analogy breaks:** any two numbers compare, but not any two functions. n and n^(1 + sin n) are neither O nor Ω of each other: the exponent keeps swinging between 0 and 2.
- **Notation inside an equation** (e.g. $2n^2 + \Theta(n) = \Theta(n^2)$): on the left it means "for every function in the set", on the right "there is one". Each "=" in a chain may lose detail, never invent it.
- **f + o(f) = Θ(f)**: lower order terms don't change the class, stated so it can be checked ($c_1 = 1$, $c_2 = 2$).

Facts to carry (lg = $\log_2$)

- **lg(n!) = Θ(n lg n)**. Upper: n! ≤ n^n. Lower: the top half of the factors are each ≥ n/2, so n! ≥ (n/2)^(n/2). This is why comparison sorting needs about n lg n comparisons.
- **n^(1/lg n) = 2** for every n > 1: a constant in disguise.
- **lg*(n)** = how many times you take lg before reaching 1. lg*(2^65536) = 5. Grows more slowly than lg lg n.
- **Polylog < polynomial < exponential**: $\lg^k n = o(n^\varepsilon)$ for any ε > 0, and $n^k = o(b^n)$ for any b > 1. Between polylog and polynomial live things like 2^√(2 lg n).

Every Θ proof has the same shape

An upper bound, a lower bound, and one threshold past which both hold: produce c_1, $c_2 > 0$ and n_0. Often the lower bound is where the idea is.

In the building (step 10): the basement was a floor all along. n^2 alone is below $T(n) = 14n^2 + 4n + 6$ at every n, so $T(n) \geq 1 \cdot n^2$ from n = 0: $T(n) = \Omega(n^2)$. With the roof, $15n^2$ from n = 6, that pins $T(n) = \Theta(n^2)$ with $c_1 = 1$, $c_2 = 15$ and $n_0 = 6$, the later of the two thresholds.

WARM-UP

1. Show $2^{n+1} = \Theta(2^n)$ by naming c_1, c_2. 2. Show $2^{2n} \neq O(2^n)$ in one line. 3. Compute lg(8!) and 8 lg 8 and (8/2)(lg 8 − 1).

PART 4 · QUESTIONS

Tier 4: The whole family: O, Ω, Θ, o, ω

4.1 *Skill: that a bound has a direction*

> "The running time of algorithm A is at least O(n²)."

A textbook exercise asks why that sentence is meaningless. Why is it?

- ◯ **A.** It is fine, and is just an informal way of writing Ω(n²).
- ◯ **B.** O(n²) is already a ceiling, so "at least" puts a floor word in front of it and rules nothing out.
- ◯ **C.** Running times are functions, and "at least" cannot be applied to a function.
- ◯ **D.** Because O-notation describes worst-case running time, and "at least" belongs to the best case.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.2 *Skill: reading a relation from the other side*

> f and g are positive from some n onward. o and ω are the small versions of O and Ω: f = o(g) says f falls behind g by every constant factor, not merely some one of them

f(n) = o(g(n)), said "little o of g of n", says the same thing as which of these?

- ◯ **A.** g(n) = o(f(n))
- ◯ **B.** g(n) = Ω(f(n))
- ◯ **C.** g(n) = ω(f(n))
- ◯ **D.** f(n) = O(g(n)) and f(n) ≠ Θ(g(n))

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.3 *Skill: what notation inside an equation means*

> $2n^2 + \Theta(n) = \Theta(n^2)$

Asymptotic notation is allowed to appear in the middle of a formula, standing for a function nobody names. What does the equation above claim?

- ◯ **A.** Both sides denote the same collection of functions.
- ◯ **B.** However the unnamed function on the left is chosen from $\Theta(n)$, some function in $\Theta(n^2)$ can be chosen to make the two sides equal.
- ◯ **C.** There is a function in $\Theta(n)$ and a function in $\Theta(n^2)$ making the two sides equal.
- ◯ **D.** It is an abuse of notation with no agreed meaning, written for brevity.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.4 *Skill: the standard facts a chapter leaves you with*

Tick every true statement. From here on lg n means $\log_2 n$, the logarithm to base 2, which is what this subject writes by default. b and k are constants with $b > 1$ and $k \geq 1$, and ε is any constant greater than 0.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $2^{n+1} = \Theta(2^n)$
- ☐ **B.** $2^{2n} = \Theta(2^n)$
- ☐ **C.** $\lg(n!) = \Theta(n \lg n)$
- ☐ **D.** $n^{(1/\lg n)} = \Theta(1)$
- ☐ **E.** $\lg^k n = o(n^{\varepsilon})$
- ☐ **F.** $n^k = \omega(b^n)$

4.5 *Skill: ranking functions nobody has drawn for you*

Put these in order of growth, slowest growing first. Two of the names need saying first: lg*(n) counts how many times you can take a logarithm of n before the result reaches 1, and n^(1/lg n) means n raised to the power one divided by lg n. Each row is a real entry from a textbook problem that asks for thirty of them at once.

A $\sqrt{(\lg n)}$ | **B** $\lg^*(n)$ | **C** $\lg \lg n$ | **D** $2^{\sqrt{(2 \lg n)}}$ | **E** $n^{(1/\lg n)}$

F $n \lg n$ | **G** $\lg^2 n$

[1] < [2] < [3] < [4] < [5] < [6] < [7]

Write the letters in the boxes, slowest-growing / smallest first. For each neighbouring pair, can you say where they cross?

4.6 *Skill: whether any two functions can be compared at all*

```
For real numbers, exactly one of a < b, a = b, a > b holds.
```

O, Ω, Θ, o and ω read like ≤, ≥, =, < and >. Which part of that analogy breaks?

- ◯ **A.** None of it. o, Θ and ω take the three roles and the analogy is exact.
- ◯ **B.** It breaks once both functions are polynomials.
- ◯ **C.** The three-way comparison breaks: there are f and g with neither f = O(g) nor f = Ω(g).
- ◯ **D.** It breaks, and the repair is to compare the limit of f(n) / g(n) instead.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.7 *Skill: which of the plausible conjectures survive*

```
f and g are positive from some n onward
```

A textbook problem gives a list like this and asks you to prove or disprove each. Tick the ones that are true.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** f = O(g) implies g = O(f)
- ☐ **B.** f = O(g) implies g = Ω(f)
- ☐ **C.** f = O(g) implies 2^f = O(2^g)
- ☐ **D.** f = O(g) implies lg f = O(lg g), where lg g ≥ 1 and f ≥ 1 for all large n
- ☐ **E.** f = O(f²)
- ☐ **F.** f + o(f) = Θ(f)

4.8 *Skill: a witness for a bound the page never drew*

```
f(n) = lg(n!) = lg 1 + lg 2 + ⋯ + lg n, g(n) = n lg n
```

Give a c and an n_0 witnessing lg(n!) = O(n lg n). The two sides are evaluated for real, so any pair that holds counts.

c = ☐ **n_0 =** ☐

Check your pair. Show that the inequality holds at every $n \geq n_0$ (a table of a few values is evidence, not proof: say why it keeps holding).

4.9 *Skill: proving a two-sided bound*

```
f and g are never negative from some n onward
```

Build the proof that max(f(n), g(n)) = Θ(f(n) + g(n)).

LINE 1 · WHAT HAS TO BE PRODUCED

- ◯ **A.** Constants c_1, $c_2 > 0$ and an n_0 with $c_1(f(n) + g(n)) \le \max(f(n), g(n)) \le c_2(f(n) + g(n))$ for every $n \ge n_0$.
- ◯ **B.** A proof that max(f, g) is whichever of f and g is the larger.
- ◯ **C.** A single c and an n_0 with $\max(f, g) \le c(f + g)$.

LINE 2 · THE UPPER BOUND

- ◯ **A.** $\max(f, g) \le f \cdot g$, so $c_2 = 1$ against the product.
- ◯ **B.** $\max(f, g) \le f + g$, since neither is negative from some n_0 on. So $c_2 = 1$.
- ◯ **C.** $\max(f, g) \le f + g$, provided f and g are both increasing.

LINE 3 · THE LOWER BOUND

- ◯ **A.** $\max(f, g) \ge (f + g) / 2$, because the larger of two numbers is at least their average. So $c_1 = 1/2$.
- ◯ **B.** $\max(f, g) \ge f$, so $c_1 = 1$.
- ◯ **C.** $\max(f, g) \ge \min(f, g)$, so $c_1 = 1$.

LINE 4 · PUTTING IT TOGETHER

- ◯ **A.** So max(f, g) = O(f + g), and since O and Ω are transposes, Θ follows.
- ◯ **B.** So max(f, g) and f + g are never more than a factor of 2 apart, hence they are equal.
- ◯ **C.** So $(1/2)(f + g) \le \max(f, g) \le 1 \cdot (f + g)$ from n_0 where both are nonnegative, which is $\Theta(f + g)$ with $c_1 = 1/2$ and $c_2 = 1$.

One line per step. The proof counts only if every line is right.

Why anybody reaches for this

Worst case, by default

An average needs a probability distribution over inputs, and real inputs don't come from a distribution you know. A worst-case bound is a promise about *every* input. Average-case analysis is done (quicksort) but its answer depends on the assumption.

Efficient = polynomial time

A running time is **polynomial** if it is $O(n^d)$ for some constant d. The property the definition is built on: **doubling the input multiplies the time by a constant**, $c(2n)^d = 2^d \cdot cn^d$, whatever n was. Doubling the input to 2^n *squares* the time. The line is about scaling, not speed: n^{100} is polynomial and hopeless; n^(1 + 0.02 lg n) is not polynomial yet usable.

- d need not be whole: n^1.59, √n, and n log n ($\leq n^2$) are all polynomial.
- A polynomial $a_0 + a_1 n + \ldots + a_d\, n^d$ with a_d > 0 is $\Theta(n^d)$ whatever the signs of the others: each term is $\leq |a_j| n^d$ for $n \geq 1$, and a fixed-size sum of $O(n^d)$ terms is $O(n^d)$. The leading term need not be largest at every n (in $n^2 - 100n + 5000$, 5000 leads until n = 50, 100n until n = 100).
- A missing **log base** is fine (bases differ by a constant factor). A missing **exponential base** is not: $(r/s)^n$ is unbounded. At a million steps a second, 1.5^{50} steps is about 11 minutes, 2^{50} is about 36 years.

Where running times come from

Bound	Usual cause
O(log n)	throw away half of what is left each step (binary search)
O(n)	one pass, constant work per item (max, merging two sorted lists)
O(n log n)	split in half, recurse, combine in linear time (mergesort); or sort then one pass
$O(n^2)$	all pairs (closest pair, the obvious way)
$O(n^3)$	three nested loops (every pair of sets, check for a common element)
$O(n^k)$	all subsets of fixed size k
$O(2^n)$	all subsets (brute-force search)
O(n!)	all orderings / all ways to pair n with n

> **What an algorithm buys.** A stable matching instance has n! possible pairings but the propose-and-reject algorithm finds a stable one in $O(n^2)$ steps. A compact input implicitly describes a huge search space; finding structure that lets you skip enumerating it is what algorithm design is. (See the Gale-Shapley packet.)

ACTIVITY F · DOUBLING

running time	n	n^2	n^3	n^5	2^n (n: 10 → 20)
factor when input doubles					

At 10^6 steps per second, how long is 2^{50} steps? And 1.5^{50} ($\approx 6.4 \times 10^8$)?

PART 5 · QUESTIONS

Tier 5: What any of this is for

5.1 *Skill: which running time gets analysed*

Why is the worst case the one that usually gets analysed, rather than the average case?

- ◯ **A.** Because the worst case is what actually happens most of the time.
- ◯ **B.** Because an average needs a distribution over inputs, and real inputs do not come from one.
- ◯ **C.** Because average-case analysis is harder, so nobody does it.
- ◯ **D.** Because Big-O can only describe worst-case running times.

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.2 *Skill: what polynomial time is actually for*

> A running time is polynomial if it is $O(n^d)$ for some constant d that does not depend on n

Efficient is usually taken to mean polynomial time. What property of polynomial bounds is that definition built on?

- ◯ **A.** Doubling the input multiplies the running time by at most a constant, and that constant does not grow with the input.
- ◯ **B.** Polynomial-time algorithms finish in a reasonable time on real inputs.
- ◯ **C.** Polynomial time is the largest class that a computer can actually run.
- ◯ **D.** Because every polynomial is $O(n^2)$, which is fast enough for most purposes.

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.3 *Skill: the scaling property, applied*

> An algorithm runs in $c \cdot n^d$ steps, with $d = 3$

The input size doubles. The running time grows by a factor of what?

× ☐

Working:

5.4 *Skill: what counts as polynomial*

Tick every running time that is polynomial.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $O(n^{1.59})$
- ☐ **B.** $O(\sqrt{n})$
- ☐ **C.** $O(n \log n)$
- ☐ **D.** $O(n^{100})$
- ☐ **E.** $O(2^n)$
- ☐ **F.** $O(n^{(1 + 0.02 \cdot \log n)})$

5.5 *Skill: why the lower terms of a polynomial can be ignored entirely*

> $p(n) = a_0 + a_1 n + \cdots + a_d n^d$, with $a_d > 0$ and the other coefficients any sign at all

$p(n) = \Theta(n^d)$, whatever the lower coefficients are, including negative ones. What carries that?

- ◯ **A.** Each term $a_j n^j$ is at most $|a_j| n^d$ for $n \geq 1$, so the sum of $d + 1$ terms each $O(n^d)$ is $O(n^d)$.
- ◯ **B.** The negative terms only make p smaller, so they can be dropped.
- ◯ **C.** Because the limit of $p(n) / n^d$ is a_d, which is positive.
- ◯ **D.** Because $a_d n^d$ is the largest term for every n.

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.6 *Skill: where a running time comes from in the code*

O(n log n) turns up more often than almost anything else. What is the usual reason?

- ◯ **A.** Two nested loops, the outer over n items and the inner over n items.
- ◯ **B.** Splitting the input into two halves, solving each recursively, and combining in linear time.
- ◯ **C.** Sorting, and nothing else.
- ◯ **D.** A loop over n items where each step does O(log n) work by binary search.

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.7 *Skill: the cost of searching each kind of space*

These are the running times that keep turning up, each with its usual cause, and the larger ones are the cost of searching a space by brute force. Put them in order of cost, cheapest first, with k fixed at 4.

A $O(2^n)$ **B** $O(n^4)$ **C** $O(n)$ **D** $O(n!)$ **E** $O(n^3)$ **F** $O(n \log n)$ **G** $O(n^2)$ **H** $O(\log n)$

[1] < [2] < [3] < [4] < [5] < [6] < [7] < [8]

Write the letters in the boxes, slowest-growing / smallest first. For each neighbouring pair, can you say where they cross?

5.8 *Skill: when a missing base matters*

Writing O(log n) without saying which base is fine. Saying "the running time is exponential" without saying which base is sloppy. What is the difference?

- ◯ **A.** Logarithms of different bases differ by a constant factor, and exponentials of different bases do not.
- ◯ **B.** There is no difference. Both are equally sloppy.
- ◯ **C.** There is no difference. Both are equally fine, because Big-O ignores constants either way.
- ◯ **D.** Because logarithms are slow-growing, so their differences do not matter.

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.9 *Skill: what an algorithm buys you*

> n students and n schools each rank the whole of the other side, and the job is to pair them up so that no two people would both rather have each other than the partner they were given. A pairing like that is called stable. There are n! ways to pair them up at all, and a simple process where people take turns proposing and every proposal is held or turned away finds a stable one in $O(n^2)$ steps.

What is that gap an instance of?

- ◯ **A.** A compact input that implicitly describes a huge search space, and an algorithm that never enumerates it.
- ◯ **B.** Proof that the stable matching problem is easier than it looks.
- ◯ **C.** A case where the brute-force algorithm was simply implemented badly.
- ◯ **D.** A sign that n^2 is the real size of the problem, and n! was never the right count.

Why not the others? Pick the one you almost chose and say what is wrong with it.

KEEP THIS PAGE

One-page summary

The definition

$T(n) = O(f(n)) \Leftrightarrow \exists c>0\ \exists n_0\ \forall n \geq n_0 : T(n) \leq c \cdot f(n)$

- A ceiling, not a measurement. No seconds, no small inputs, no closeness.
- n_0 = first n covered ($n \geq n_0$ convention).
- Witness = one pair (c, n_0) + a check. Not unique; smallest earns nothing.
- c stretches, never bends. $c \cdot n^2$ never catches n^3.
- Lower-order terms are *paid for* by raising c, not deleted. Small share ≠ small number.
- For $14n^2 + 4n + 6$: c = 14 never; c = 15 from $n_0 = 6$; c = 18 from 2.

Growth ladder

$1 < \lg^* n < \lg \lg n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$

Code shapes

- subtract → n; divide → log n; nested → ×; sequential → + (keep the larger)
- Σ i for i < n = $n(n-1)/2 = \Theta(n^2)$

The family

- $O \leq \cdot\ \Omega \geq \cdot\ \Theta = \cdot\ o < \cdot\ \omega >$
- O(g) ⇔ transpose Ω; o ⇔ ω
- Not every pair of functions compares.
- In equations: left side ∀, right side ∃.

Proofs

- Negate: flip every quantifier, keep order, flip relation.
- Contradiction: keep c general; build breaking n from c and n_0; pass the ceiling strictly.
- Shared ceiling ≠ transitivity.
- Θ proof: c_1, c_2, one n_0.

Rewrites

- Constant factors forgiven; constants in exponents not.
- Log base: irrelevant. Exponential base: matters.
- lg(n!) = Θ(n lg n); n^(1/lg n) = 2.

Efficiency

- Worst case by default (no distribution needed).
- Polynomial = $O(n^d)$, constant d; doubling n costs $\times 2^d$.
- Exponential: doubling n squares the cost.

ANSWER KEY

Answer key: activities

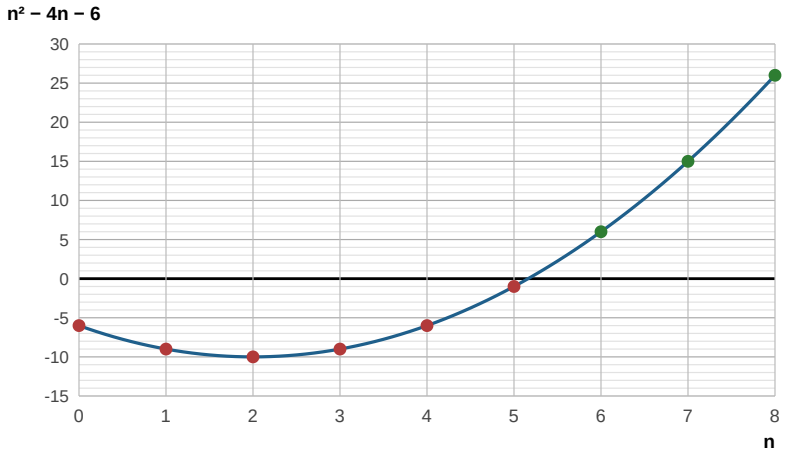
Try everything before you look.

Activity A

n	T(n)	14n²	15n²	15n² ≥ T?	n²	4n+6	n² ≥ 4n+6?
1	24	14	15	no	1	10	no
2	70	56	60	no	4	14	no
3	144	126	135	no	9	18	no
4	246	224	240	no	16	22	no
5	376	350	375	no	25	26	no
6	534	504	540	yes	36	30	yes
7	720	686	735	yes	49	34	yes
8	934	896	960	yes	64	38	yes

1. Never: it cancels to $0 \geq 4n + 6$, and $4n + 6$ is positive at every n. 2. n = 6; the columns flip together because $15n^2 \geq T(n)$ is the same inequality as $n^2 \geq 4n + 6$. 3. No: the share is small but the amount is 50 and growing; $14n^2$ is short by exactly that much at every n.

Activity B



Values: 0→-6, 1→-9, 2→-10, 3→-9, 4→-6, 5→-1, 6→6, 7→15, 8→26. Negative from 0 through 5, crosses zero at $n = 2 + \sqrt{10} \approx 5.16$. n is a whole number and the rule covers every $n \geq n_0$, so the first whole n past the crossing, 6, is n_0.

Activity C

c	14	15	16	17	18	50
n_0	none	6	3	3	2	0

Bigger constant, earlier threshold. Every pair is an equally good witness; nothing rewards the tightest.

Activity D

Halving loop: n = 1 → 0, 2 → 1, 8 → 3, 10 → 3, 100 → 6, 1000 → 9. About $\log_2 n$: $O(\log n)$.
Triangular loop: n = 1 → 0, 2 → 1, 3 → 3, 4 → 6, 5 → 10, 10 → 45, matching $n(n-1)/2$. $\Theta(n^2)$.

Activity E

n	f	g	1·g	2·g	3·g	4·g
0	0	3	3 ✓	6 ✓	9 ✓	12 ✓
1	1	1	1 ✓	2 ✓	3 ✓	4 ✓
2	4	1	1 ×	2 ×	3 ×	4 ✓
3	9	3	3 ×	6 ×	9 ✓	12 ✓
4	16	7	7 ×	14 ×	21 ✓	28 ✓
5	25	13	13 ×	26 ✓	39 ✓	52 ✓
6	36	21	21 ×	42 ✓	63 ✓	84 ✓

1. c = 3 fails only at n = 2 (the middle, not the start): $n_0 = 3$, since $3g - f = (2n - 3)(n - 3)$ is negative only between 1.5 and 3. 2. c = 2 fails at 2, 3, 4: $n_0 = 5$. 3. $4g - f = 3n^2 - 12n + 12 = 3(n - 2)^2 \geq 0$, so c = 4 works from $n_0 = 1$ (touching at n = 2), and nothing below 4 survives n = 2. 4. c = 1 cancels to $3n \leq 3$, true at n = 1 only: every larger n fails.

Part 4 warm-up

1. $2^{n+1} = 2 \cdot 2^n$, so $c_1 = c_2 = 2$ (any $c_1 \leq 2 \leq c_2$). 2. $2^{2n} / 2^n = 2^n$, which passes every constant. 3. lg(8!) = lg 40320 ≈ 15.3; 8 lg 8 = 24; (8/2)(lg 8 − 1) = 8. So 8 ≤ 15.3 ≤ 24, as the two bounds say.

Activity F

×2, ×4, ×8, ×32; for 2^n going from n = 10 to 20 the time is multiplied by $2^{10} = 1024$, i.e. squared ($2^{20} = (2^{10})^2$), and the factor itself grows with n. $2^{50} / 10^6$ s ≈ 1.13×10^9 s ≈ 36 years; $1.5^{50} / 10^6$ ≈ 638 s ≈ 11 minutes.

Answer key: questions

1.1

Answer: **D**

- **A** × Big-O carries no scale. c is unnamed and could be 0.001 or 5000, and nothing says T(n) gets anywhere near the ceiling: the constant function T(n) = 6 is $O(n^2)$ as well.
- **B** × Two faults at once. $O(n^2)$ is only a ceiling, so this algorithm might really be O(n) as well. And even with true growth rates, constants decide real inputs: 1000n loses to n^2 only once n passes 1000.
- **C** × That is a floor, which is Ω, not O. Big-O bounds from above only. A claim that does both at once is Θ.
- **D** ✓ That is the entire content of the claim. Some c exists, some n_0 exists, and from n_0 on the ceiling holds. No promise about seconds, none about small inputs, and none that T(n) is close to n^2.

Takeaway. Big-O is an existence claim about two functions. Everything people usually want from it, speed, tightness, worst case, has to be asked for separately.

1.2

Answer: **B**

- **A** × That is f(n) + c, not c·f(n). Adding lifts every point by the same amount. Multiplying lifts each point in proportion to how high it already is, which is why the effect grows with n and why it can catch a curve that is pulling away.
- **B** ✓ That is the whole mechanism of the definition. c is a vertical stretch and nothing else. It cannot bend a curve, move where it is steep, or change which function eventually wins. All it can do is lift the whole thing until it clears T(n) from some point on.
- **C** × No c does that. $c \cdot n^2 \geq n^3$ rearranges to $c \geq n$, which fails the moment n passes c. Growth rate belongs to the shape, and constants cannot touch it. That is exactly why Big-O is free to ignore them.
- **D** × Raising c moves the crossing earlier, and it moves nothing sideways. On this page's own $T(n) = 14n^2 + 4n + 6$, compare c = 15 with $n_0 = 6$ against c = 18 with $n_0 = 2$: more constant, earlier crossing.

Takeaway. The constant is a lift, not an accelerator. Everything Big-O forgives, it forgives because a lift is all c can do.

1.3

Answer: **D**

- **A** × It would be, if the roof meant one thing throughout. It does not, and the page says so rather than leaving you to notice: the meaning switches once, and the switch is what the next option describes.
- **B** × Then 4n and 6 would never have been on the roof at all. But the opening steps are about naming the parts of T(n), and 6 is one of them. Membership, not size.
- **C** × 6 is a term of T(n) at every n. Nothing about T(n) changes when n grows. What changes is only which term dominates the sum, and that is a fact about the sum, not about the terms.
- **D** ✓ Under the first reading n^2 would belong on the roof, since it is part of the leading term. Under the second it does not, because n^2 alone is below T(n) at every n. The two readings genuinely contradict each other, which is why naming the switch is not optional.

Takeaway. Two different questions wear the same picture: what is T(n) made of, and what sits above T(n). Keeping them apart is most of what the definition asks you to hold.

1.4

Answer: **C**

- **A** × It does not become small. At n = 1000 it is 4006, and it grows without limit. What shrinks is its share of T(n), and a small share is not a small number. $14n^2$ alone falls short of T(n) by exactly 4n + 6, at every n, forever.
- **B** × The definition never mentions terms. It says some c and some n_0 exist with $T(n) \le c \cdot f(n)$ at every n from n_0 on. Dropping lower order terms is something you derive from that, not a rule written into it, and treating it as a rule is exactly what makes c = 14 look reasonable.
- **C** ✓ This is a payment, not a dismissal. The terms are still there and still positive. What makes them droppable from the name is that the constant, which the definition hands you for free, is large enough to absorb them.
- **D** × True, and not sufficient. $14n^2$ is also smaller than T(n) at every n, which is precisely why $14n^2$ is not a bound. Being the smaller part of a sum is no licence to delete it.

Takeaway. The terms are outgrown, not killed. What lets you stop naming them is the constant, and that is why c = 15 and not 14.

1.5

Answer: **A**

- **A** ✓ c = 14 is the one value that fails structurally rather than by being too small: it leaves nothing over to pay for what was dropped. Every c above 14 works eventually, because the spare $(c - 14)n^2$ outgrows 4n + 6.
- **B** × The share does go to zero, and that part is true. But the inequality is not about shares. Cancel $14n^2$ from both sides of $14n^2 \ge 14n^2 + 4n + 6$ and what is left is $0 \ge 4n + 6$, which is false at every n. A share fading to nothing is not a quantity reaching zero.
- **C** × Check n = 1000. $14n^2$ is 14,000,000 and T(1000) is 14,004,006, still short by 4006, which is exactly 4n + 6. The shortfall is not a small-n artefact. It is the size of what you threw away, and it grows.
- **D** × True here, but not a rule. It holds because the remainder 4n + 6 is positive. Take $T(n) = 14n^2 - 4n$ instead and c = 14 works from n = 0 on, with nothing to spare. What decides it is the sign of what is left over, not the coefficient.

Takeaway. Raising c is a payment for the terms you stopped naming. c = 14 is the one constant that pays nothing.

1.6

True: **D, E, F** · False: A, B, C

- **A** × $n_0 = 3$ asks about every n from 3 up, and it fails at once: at n = 3, 15·9 = 135 against T(3) = 144.
- **B** × Two short. $n_0 = 4$ asks about every n from 4 up, and at n = 4, 15·16 = 240 against T(4) = 246.
- **C** × One short, and by a single value of n. $n_0 = 5$ asks about every n from 5 up, and n = 5 is the one that still fails: 15·25 = 375 against T(5) = 376.
- **D** ✓ The smallest that works. n = 5 is the last failure, and the rule asks about every n at or above n_0, so n_0 names the first n that holds rather than the last that fails.
- **E** ✓ Every n_0 above the smallest also works, because a larger n_0 asks about fewer values of n. A weaker witness than $n_0 = 6$, and an equally valid one, where a witness is just the pair of numbers, one c and one n_0, that makes the definition come true.
- **F** ✓ The definition asks that some n_0 exists, never that you found the smallest, so $n_0 = 100$ proves the same statement as $n_0 = 6$.

Takeaway. n_0 is not unique, and neither is c. The definition is an existence claim: one witness is enough, and the tightest witness earns no extra credit.

1.7

Answer: $\mathbf{n_0 = 2}$

$18n^2 \ge 14n^2 + 4n + 6$ is $4n^2 - 4n - 6 \ge 0$, which is false at n = 1 (4 − 4 − 6 = −6) and true from n = 2 on (16 − 8 − 6 = 2). So the first n that holds is 2, and $n_0 = 2$.

- **If you wrote 1:** $n_0 = 1$ asks about every n from 1 up, and n = 1 already fails: 18·1 = 18 against T(1) = 24. So n_0 has to be at least 2.
- **If you wrote 6:** That is the n_0 that goes with c = 15. A larger constant opens the margin sooner, so its n_0 drops: at n = 2, 18·4 = 72 against T(2) = 70, already clear.

- **If you wrote 3:** One too far. n = 2 is already clear, at 72 against 70, so the threshold does not have to wait until 3. n_0 names the first n that holds, not the first comfortable one.

Takeaway. c and n_0 trade against each other: 15 needs $n_0 = 6$, 16 needs 3, 18 needs 2, and c = 50 needs only $n_0 = 1$. Every one of those pairs is an equally good witness that $T(n) = O(n^2)$.

1.8

Order: **D < C < E < A < B**

1. **D · 6**: A constant. It never grows, so everything else passes it and nothing ever falls back below it.
2. **C · 4n + 6**: Linear. The + 6 is invisible to the ordering, because a constant added to a growing function stops mattering to which is bigger.
3. **E · n^2**: Quadratic, and it passes 4n + 6 at n = 5.16. That crossing is where $n_0 = 6$ comes from: n = 6 is the first whole number past the moment the shape outgrows the terms that were dropped.
4. **A · 14n^2**: Fourteen times n^2, so it sits above n^2 at every n and the ranking is safe. But it never pulls away: $14n^2 / n^2$ is 14 forever, so these two are the same growth class, ordered here only by size.
5. **B · n^3**: Cubic, and it reaches $14n^2$ at exactly n = 14, where both are 2744, then leaves it behind. The constant 14 buys fourteen values of n and not one more.

Takeaway. Two of these, n^2 and $14n^2$, are the same growth class: a constant separates them and nothing else. n^3 and $14n^2$ are not, and that difference is the only one Big-O can see.

1.9

True: **A, B, D, F** · False: C, E

- **A** ✓ With c = 15 and $n_0 = 6$. This is the tight one, which is why it is the one worth saying.
- **B** ✓ A ceiling does not have to be low. c = 1 and $n_0 = 15$ work: at n = 15, n^3 is 3375 against T(15) = 3216, and n = 14 is the last failure, at 2744 against 2806. Weaker than $O(n^2)$, but not false.
- **C** × For any fixed c, the line cn loses to $14n^2$ once n passes c, and the gap only widens. Big-O forgives any constant, but no constant turns a line into a parabola.
- **D** ✓ Stronger than $T(n) = O(n^2)$, because Θ pins T(n) from both sides at once: $14n^2 \le T(n) \le 15n^2$ for every n from 6 on. Only n^2 can carry a Θ here.
- **E** × This is where O and Θ come apart. $T(n) = O(n^3)$ is true, but the floor fails: $T(n) / n^3$ goes to zero, so no c keeps $c \cdot n^3$ underneath T(n). Loose ceilings are allowed. Loose Θ is not.
- **F** ✓ Same reason as n^3, and the same c even: $\log_2 n$ is at least 1 from n = 2, so $15n^2 \log_2 n$ is at least $15n^2$, which already covers T(n) from n = 6 on. A valid ceiling, just a needlessly high one.

Takeaway. Four of these are true at the same time about the same function, because Big-O asks whether a ceiling exists and never which ceiling is lowest.

2.1

Answer: **A**

- **A** ✓ O(1) says a constant ceiling exists: some c with $T(n) \le c$ for all large n. It does not say the cost is one step, that it is the same every time, or that it is small.
- **B** × O(1) means bounded by a constant, and the constant is never named. A routine that always takes 40,000 steps is O(1).
- **C** × Not on any input you will ever run. An O(1) with a constant of 40,000 loses to an O(log n) with a constant of 1 until log n passes 40,000. Asymptotic claims compare shapes, not runs.
- **D** × A bound is not a measurement. A lookup that takes anywhere between 1 and 500 steps depending on collisions is still O(1), because 500 is a constant.

Takeaway. Every Big-O class has this shape. The class names how the cost responds to n, and says nothing at all about how big the cost is.

2.2

Answer: **B**

- **A** × This welds together two unrelated distinctions. O and Ω are upper and lower bounds on one chosen function. Worst and best case are different choices of which function to bound. You can put a lower bound on the worst case, and that is exactly what a proof that comparison sorting is Ω(n log n) does.
- **B** ✓ Worst case, best case and average case are three different functions of n. Each can be bounded above, so quicksort's worst case is $O(n^2)$ and its best case is O(n log n), and both are Big-O statements. Nothing in the definition mentions cases at all.
- **C** × The definition contains no maximum and no inputs, only two functions of n. Taking the maximum over inputs of size n is what produces the worst case function, before any bound has been applied to it.
- **D** × It is about neither on its own. Big-O is silent on which case you picked, which is why the case has to be stated alongside it.

Takeaway. Two independent choices get collapsed into one all the time: which function you are describing, and which side you are bounding it from.

2.3

Answer: **C**

- **A** × That is the count if k went down by one each pass. Dividing is the whole difference: n = 1000 takes nine passes, not a thousand.
- **B** × The count really is $\log_2 n$, but inside Big-O the base folds into c, because $\log_2 n = \log n / \log 2$. Taking thirds instead of halves changes the count by a factor of about 1.58, and no Big-O statement can see a factor.
- **C** ✓ Each pass halves k, so after t passes k is about $n / 2^t$, and the loop ends when that reaches 1. That makes t about $\log_2 n$: nine passes for n = 1000.
- **D** × The body is O(1). The loop is not. The bound counts how many times the body runs, and that still depends on n, just very slowly.

Takeaway. What a loop does to its counter decides the shape: subtract and you get n, divide and you get log n.

2.4

Answer: **B**

- **A** × The bound is right and the count is not. The inner loop runs i times, not n, so the total is $0 + 1 + \ldots + (n - 1) = n(n - 1) / 2$. The answer survives the miscount because constants do not matter, which is a habit worth distrusting: it works here and hides real errors elsewhere.
- **B** ✓ The count is exactly $n(n - 1) / 2$. The one half is a constant factor, so it disappears into c: this is $\Theta(n^2)$, and $O(n^2)$ is both correct and tight. Doing half the work does not change the growth class.
- **C** × Tightness is not about running fewer iterations, it is about the shape. $n(n - 1) / 2$ divided by n^2 settles at one half, a constant, so n^2 is exactly right. A bound is loose only when that ratio goes to zero, as it would against n^3.
- **D** × Shorter is not the same as logarithmically shorter. This inner loop shrinks by a fixed amount each pass, so the totals are $0 + 1 + 2 + \ldots + (n - 1)$, a sum that steps up by one each time and comes to about $n^2 / 2$. A logarithm appears when the size is divided, not decremented.

Takeaway. Constant factors are invisible to Big-O whether they help you or hurt you, so halving the work leaves the bound exactly where it was.

2.5

Answer: **C**

- **A** × That is the answer if the passes were nested, one running inside the other. They run one after the other, so the costs add rather than multiply.
- **B** × True, and it names the same set of functions as $O(n^2)$, so it is exactly as tight. What is wrong with it is that it is unsimplified: the first term contributes nothing, and leaving it in hides which stage the bound is about.
- **C** ✓ A sum is bounded by a constant times its larger part: $n \log n + n^2 \le 2n^2$ from $n = 1$ on. This is why the slowest stage names the whole routine, and why speeding up the sort here buys nothing asymptotically.
- **D** × Reversed. n^2 is above $n \log n$ at every n from 1 on, since $n^2 \ge n \log n$ cancels to $n \ge \log n$, and the gap only widens. Sorting feels expensive because it usually is the slow stage, which is what makes this the easy one to get backwards.

Takeaway. Sequential stages add and take the larger; nested stages multiply. Which one you are looking at is a fact about the code, not about the bounds.

2.6

Order: **F < C < A < B < D < G < E**

1. **F · log n**: Grows without bound, but slower than any positive power of n. Doubling n adds one step.
2. **C · √n**: The square root of n, which is n^0.5, a power of n. So it grows faster than every logarithm and more slowly than n.
3. **A · n**: Linear. Doubling n doubles the work, which is the baseline everything else is judged against.
4. **B · n log n**: Just above linear, and not by much: the log factor is 20 at n = a million. Close enough to n that the difference is often affordable, and far enough to be a real separation.
5. **D · n^2**: Quadratic. $n \log n / n^2$ goes to zero, so this is a genuine step up and not a constant apart.
6. **G · 2^n**: Exponential, and it eventually passes every polynomial, including n^{100}. Where polynomials multiply the work when n doubles, this squares it.
7. **E · n!**: n factorial: n times every whole number below it, so 5! is 120. Faster still than 2^n, because $n! / 2^n$ grows without bound: past n = 2 each new factor is larger than 2.

Takeaway. The gaps here are not the same size. n and n log n are almost neighbours, while n^2 and 2^n are not on speaking terms, and Big-O flattens both differences into the same word: not equal.

2.7

Answer: **$n_0 = 137$**

$4n^2 \ge 3n^2 + 100n + 5000$ is $n^2 - 100n - 5000 \ge 0$, whose root is $50 + \sqrt{7500}$, about 136.6. So the first whole n that works is 137.

- **If you wrote 136:** That is the last n that fails, not n_0. n_0 names the first n that holds, because the rule reads for every n at or above n_0. At n = 136, 4·18,496 = 73,984 against T(136) = 74,088.
- **If you wrote 6:** That is the n_0 from the walkthrough example, and this function is nothing like it. Here 100n + 5000 dominates for a long time: at n = 10, $4n^2$ is 400 against T(10) = 6300.
- **If you wrote 1:** Far too early. At small n the lower order part is the whole function: T(1) = 5103 against 4·1 = 4.

Takeaway. An n_0 of 137 is exactly as valid as an n_0 of 6, and an n_0 of a billion would be too. The definition asks only that one exists, which is the price of a claim that says nothing about small inputs.

2.8

True: **A, C, E** · False: B, D, F

- **A** ✓ 2^{n+1} is $2 \cdot 2^n$, so c = 2 and $n_0 = 1$ settle it. An offset in the exponent is a constant factor in disguise.
- **B** × 2^{2n} is 4^n, and $4^n / 2^n$ is 2^n, which passes every fixed c. A factor in the exponent changes the base, and no constant survives that.

- **C** ✓ $\log(n^2) = 2 \log n$, so $c = 2$ works. This is also why nobody writes the base of a logarithm inside Big-O: changing base only multiplies by a constant.
- **D** × $(\log n)^2 / \log n$ is $\log n$, which is unbounded. Squaring the logarithm is not taking the logarithm of a square, and those two sit either side of this line.
- **E** ✓ $n!$ is a product of n factors, each at most n, so $n! \le n^n$ with $c = 1$ and $n_0 = 1$.
- **F** × $n^n / n!$ passes any constant: it is about 2756 at $n = 10$ and about $4 \cdot 10^7$ at $n = 20$. That is what makes $n! = O(n^n)$ a real, one-directional bound rather than a pair of bounds.

Takeaway. Constants are forgiven anywhere they behave like constants. Inside an exponent they do not, which is the one place this intuition breaks.

2.9

True: **A, B, D, F** · False: C, E

- **A** ✓ With $c = 1$. Adding a lower order term to the ceiling only raises it.
- **B** ✓ With $c = 2$ from $n = 1$ on. Both directions hold for this pair, so these two are Θ of each other: the same function up to a constant.
- **C** × $(n \log n) / n$ is $\log n$, which is unbounded, so no fixed c can hold.
- **D** ✓ With $c = 1$ from $n = 2$. Only this direction holds, which is what makes n and $n \log n$ a real separation rather than a Θ.
- **E** × Every exponential eventually beats every polynomial, however large the exponent. n^{100} leads until around $n = 1000$, and Big-O does not look at anything before n_0.
- **F** ✓ Read the other way round. The crossing is absurdly late, which is exactly as irrelevant as it sounds: the definition asks only that one exists.

Takeaway. $f = O(g)$ is one-directional. When it happens to run both ways the pair is Θ, and when it runs one way only you have found a real difference in growth.

3.1

Answer: **A**

- **A** ✓ c and n_0 are chosen once, and then every n from n_0 on has to obey. That order is the whole definition: the constant cannot be adjusted as n grows, which is exactly what makes the claim say anything.
- **B** × A real statement, and a much stronger one: every constant works eventually, however small. That is what $f(n) = o(g(n))$ says, and n^2 is not $o(n^2)$ while it certainly is $O(n^2)$.
- **C** × This asks only that the inequality keep happening, not that it keep holding. A function that dips under the ceiling infinitely often and spends the rest of its life above it satisfies this and is not $O(g)$.
- **D** × Moving $\exists c$ inside $\forall n$ lets the constant be picked after n is known, so a different c is allowed at every n. Every pair of positive functions satisfies that, and a condition nothing can fail is testing nothing.

Takeaway. Quantifiers are not decoration on the inequality, they are the claim. Read left to right, each one is chosen knowing only what stands to its left. The last one is also the place conventions differ: this page writes $\forall n \ge n_0$, following Kleinberg and Tardos and CLRS, so n_0 is the first n the claim covers. Some books and courses write $\forall n > n_0$ instead, making n_0 the last n allowed to fail. Nothing about the claim changes, but every "smallest n_0" answer shifts by one, so read which one a question means before answering it.

3.2

Any pair that really holds is correct. There is nothing to match against.

Any $c \ge 4$ works with $n_0 = 1$, because $4g(n) - f(n) = 3(n - 2)^2$ is a square and never negative. Smaller constants work too, with a threshold to pay for them: $c = 3$ needs $n_0 = 3$, $c = 2$ needs $n_0 = 5$, and $c = 1$ needs an n_0 that does not exist.

- **If your c was 1 or less:** c = 1 asks $n^2 \le n^2 - 3n + 3$, which cancels to $3n \le 3$. That holds at n = 1 and at no n above it, and the failures never stop, so no n_0 can rescue it. Anything smaller is worse.
- **If your c was between 1 and 4 and the pair failed:** Every constant strictly between 1 and 4 does work eventually and has to buy a threshold to do it, because 4 is the smallest that survives n = 2, where g(2) = 1 against f(2) = 4. c = 3 needs $n_0 = 3$, and c = 2 needs $n_0 = 5$, since it also fails at n = 3 and n = 4.
- **If you gave c = 4, n_0 = 1:** That is the tightest pair there is, and it needs no real threshold at all: $4g(n) - f(n) = 3n^2 - 12n + 12 = 3(n - 2)^2$, a square. The two curves touch at n = 2 and never cross.

Takeaway. A proof from the definition is two numbers and a check, not a rearrangement. Nothing asks for the smallest c, which is why there is no single answer to compare yours against.

3.3

Answer: **c = 4**

$4g(n) - f(n) = 3n^2 - 12n + 12 = 3(n - 2)^2$, zero at n = 2 and positive at every other n. At n = 2 the two sides are both 4, so c = 4 is exactly tight and nothing below it survives that single point.

- **If you wrote 1:** c = 1 is $n^2 \le n^2 - 3n + 3$, which cancels to $3n \le 3$. It holds at n = 1 and nowhere above.
- **If you wrote 2:** c = 2 fails at n = 2, 3 and 4: at n = 3, $2 \cdot g(3) = 6$ against f(3) = 9. It works from n = 5, which is a threshold of 5, not 1.
- **If you wrote 3:** One short, and by one value of n. c = 3 clears every n except n = 2, where g(2) = 1, so $3 \cdot g(2) = 3$ against f(2) = 4.
- **If you wrote 5:** c = 5 works, and so does every constant above 4. The question asks for the smallest, and 4 already holds everywhere: $4g(n) - f(n) = 3(n - 2)^2$.

Takeaway. The entire difference between c = 3 and c = 4 is one value of n. A constant is not read off the shape of the function, it is paid to the worst point.

3.4

Answer: **n_0 = 3**

$3g(n) - f(n) = 2n^2 - 9n + 9 = (2n - 3)(n - 3)$, negative only strictly between n = 1.5 and n = 3. The only whole number in that gap is n = 2, so n = 2 is the single failure and the inequality holds at every n from 3 up: $n_0 = 3$.

- **If you wrote 0:** $n_0 = 0$ asks about every n from 0 up. n = 0 and n = 1 are both fine, at $0 \le 9$ and $1 \le 3$, but n = 2 is not: g(2) = 1, so $3 \cdot g(2) = 3$ against f(2) = 4.
- **If you wrote 2:** One short, and for a reason worth seeing: n = 2 is the only n anywhere that fails, and $n_0 = 2$ asks about $n \ge 2$, so it asks about exactly the bad one.
- **If you wrote 4:** One too far. n = 3 already holds, and holds exactly: $3 \cdot g(3) = 9$ against f(3) = 9, level rather than clear, which is what $\le$ allows.
- **If you wrote 5:** That works, and so would 100. The question asks for the smallest, which is the first n of the good stretch and not a safe distance into it.

Takeaway. The values that fail are not always the small ones. Here n = 0 and n = 1 hold, n = 2 fails, and everything from 3 up holds: n_0 is the start of the good stretch, and the good stretch does not have to start at the beginning.

3.5

Answer: **A**

- **A** ✓ A shared ceiling relates each function to the ceiling and to nothing else. n^2 and n are both $O(n^2)$, and n^2 is not O(n). The conclusion still needs its own witness, some c and n_0 with $n^2 \le c \cdot g(n)$, and the argument never produces one.
- **B** × The first step is standard and true. The second does not follow from it: additivity gives a statement about g and n^2, and the conclusion is a statement about f and g. No step in between relates them.

- **C** × Additivity is fine. Two functions under $c_1 \cdot n^2$ and $c_2 \cdot n^2$ have a sum under $(c_1 + c_2) \cdot n^2$, and one more constant is always affordable. Multiplying the functions is what raises the exponent.
- **D** × It does go negative, and Big-O is normally set up for functions that do not, so the wording is loose. But that is not the break: replace s by its absolute value and the final step still does not follow.

Takeaway. Transitivity runs in one direction: f = O(g) and g = O(h) give f = O(h). Two functions sharing a ceiling is not transitivity, and mistaking it for transitivity is the usual way a proof of f = O(g) ends up proving something else.

3.6

Answer: **A**

- **A** ✓ Each constant has to be broken by this argument, and the proof only broke one. Written properly the constant stays as c all the way to the witness, which is then built out of c, so that one page covers all of them at once.
- **B** × It cannot, and that is the content of the claim. $n \log_2 n \le 10n$ holds for every n up to 1024 and fails after; $n \log_2 n \le c \cdot n$ holds up to 2^c for any c. Each constant buys a longer stretch, so there is no single threshold covering them all, and the c = 1 argument, which fails from n = 2, is a different argument.
- **C** × n is at least n_0, and n_0 can always be taken to be at least 1, so the division is fine. Nothing is asserted at n = 0 anyway.
- **D** × That is how a proof by contradiction works. The assumption is pushed until it says something plainly false, and where the falsehood lands is not the issue.

Takeaway. A proof by contradiction is only ever as general as what it assumed. Fixing the constant on the first line is how a correct-looking page ends up proving a special case.

3.7

True: **A, B, C** · False: D, E, F

- **A** ✓ c = 4 and $n_0 = 1$, since $4g(n) - f(n) = 3(n-2)^2$ is never negative.
- **B** ✓ c = 1 and $n_0 = 1$. $g(n) \le n^2$ is $3 \le 3n$, true from n = 1 on. The one n where g is the larger is n = 0, which the threshold excludes.
- **C** ✓ Both bounds above hold at once, which is what Θ is. Neither function pulls away: g(n) / f(n) goes to 1.
- **D** × f − g is 3n − 3, which is O(n) and not O(1). The two are the same growth class and still drift apart without limit, the same lesson as $14n^2$ and 4n + 6 on the walkthrough.
- **E** × g grows like n^2. For any c the line c·n is passed and stays passed: g(100) = 9703 needs c of at least 98, and g(1000) = 997,003 needs 998.
- **F** × The first half is true and the second half is not something anybody can evaluate. f(n) = O(g(n)) is a claim about a pair of functions, never about a value of n, and the n_0 inside it exists precisely so that finitely many bad points cost nothing. One failing n cannot make the claim fail, because the claim was never made there.

Takeaway. Same class, both directions, and a gap between them that grows forever. Θ pins the shape and never the distance.

3.8

True: **A, B, C** · False: D, E, F

- **A** ✓ Each ∃ becomes ∀, the ∀ becomes ∃, the order is kept and the inequality flips. This is the one to write: whatever c and n_0 someone offers, there is an n at or past n_0 that breaks them.
- **B** ✓ The same statement one step earlier in the working. Pushing ¬ through the last ∀ turns this into the ∃n form, so both are correct and a question that accepts only one of them is marking handwriting.
- **C** ✓ Two ∀ standing next to each other can be swapped without changing the meaning, because neither depends on the other. Order matters across a change of quantifier, not within a run of the same one.
- **D** × Only the inequality was negated and the quantifiers were left alone. What this says is that f eventually stays above some fixed multiple of g, which is a lower bound, and lower bounds and upper bounds can both hold at once.

- **E** × Every quantifier flipped except the middle one, which was left as ∃. The result says f eventually beats every multiple of g and stays there, far stronger than merely failing to be O(g).
- **F** × This follows from the negation and is not equal to it. f(n) = 2g(n) breaks the ceiling c = 1 at every n and is still O(g) with c = 2. A consequence of a statement is not the statement.

Takeaway. Negation walks along the line: every quantifier turns into the other kind, nothing changes places, and the relation at the end flips. What survives untouched is the order.

3.9

Lines: **1A · 2C · 3A · 4B**

LINE 1 · THE ASSUMPTION

- **A** ✓ The definition, written out. c and n_0 exist but are not ours to choose, and everything after this has to work for whichever ones they turn out to be.
- **B** × This is the assumption with c quietly set to 1. Nothing in the definition says the constant is 1, so a contradiction from here rules out c = 1 and leaves every other constant standing.
- **C** × The quantifiers are swapped. As written this assumes T(n) = o(n), which is stronger, and contradicting a stronger assumption tells you nothing about the weaker one you meant.

LINE 2 · THE REARRANGEMENT

- **A** × That is the division carried out on the c = 1 version. With a general c on the right, dividing by n leaves c behind, not 0.
- **B** × Correct algebra going nowhere. n is still in it, so the contradiction would have to come from the size of n rather than from the constant, which is the long way round for no gain.
- **C** ✓ Dividing by something positive keeps the direction of the inequality, and it strips the proof down to the one thing in dispute: whether a logarithm can be held under a constant.

LINE 3 · THE N THAT BREAKS IT

- **A** ✓ The one step doing real work. It has to clear n_0, because nothing was ever claimed below it, and it has to clear c by a margin, because ≤ is what is being broken.
- **B** × $\log_2 n = c$ satisfies $\log_2 n \le c$. Reaching the ceiling exactly is not breaking it, and this n might also sit below n_0, where nothing was claimed.
- **C** × Nothing licenses that. n_0 could be 3 while c is 1000, and $\log_2 4 = 2$ is comfortably under. The n that breaks it has to be built out of c, not out of the threshold.

LINE 4 · THE CONCLUSION

- **A** × The second sentence is the claim, asserted rather than shown. One constant failing is one constant failing, and the proof was arranged so that they never have to be checked one at a time.
- **B** ✓ The contradiction is with the assumed pair, whatever it was, so what falls is the existence of any such pair. That is precisely the negation of the definition.
- **C** × True about T(n), and an answer to a different question. No lower bound follows from this argument, and the thing being proved was that an upper bound fails.

Takeaway. Everything that goes wrong in a disproof by contradiction goes wrong in the same place: the assumption hands you a constant, and the argument has to survive whichever one it was.

4.1

Answer: **B**

- **A** × It is not a loose way of saying $\Omega(n^2)$, it is a sentence with no content, and the two come apart on exactly the algorithms anyone cares about: an O(n) algorithm satisfies "at least $O(n^2)$" under this reading and is certainly not $\Omega(n^2)$.
- **B** ✓ Take any running time at all. The constant function 6 is $O(n^2)$, and every algorithm that does anything takes at least 6 steps, so the sentence is satisfied by everything. A claim satisfied by everything is not a weak claim, it is an empty one.

- **C** × It applies perfectly well: T(n) ≥ g(n) is a fine statement about two functions. The trouble is what stands on the right, which is not a function but a whole collection of them, and one bounded from above at that.
- **D** × O-notation applies to whatever function it is pointed at: worst case, best case, average case, the number of comparisons. Which running time is being bounded is a separate thing to say, and saying it would not rescue this sentence.

Takeaway. O is a ceiling and Ω is a floor. Sentences that mix the two are the most common way a statement about running time ends up asserting nothing at all.

4.2

Answer: **C**

- **A** × The same relation pointed the other way, which is the opposite claim. o behaves like <, and swapping the two sides of a < does not leave it alone.
- **B** × That is f = O(g) read from the other side, which is what transpose means, and it is weaker: it allows the two to grow at the same rate. o says f falls behind g by any constant factor you care to name, so its transpose has to say the matching thing.
- **C** ✓ Transpose symmetry. Reading the same relation from the other side turns o into ω exactly as it turns O into Ω, so f asymptotically smaller than g is g asymptotically larger than f.
- **D** × True of the functions that usually come up and not equivalent in general. Let f be n at even n and 1 at odd n, against g = n: f is O(g), f is not Θ(g), and f is not o(g) either, because the ratio keeps returning to 1.

Takeaway. The five relations pair up: O with Ω, o with ω, and Θ with itself. Θ being its own transpose is why it is the one that reads like equality.

4.3

Answer: **B**

- **A** × They do not. $2n^2 + \Theta(n)$ is a far smaller collection than $\Theta(n^2)$, which contains $7n^2$ and $n^2/100$. The equation is deliberately one-way, and the right-hand side is the coarser of the two.
- **B** ✓ Notation on the left is read as "for every", notation on the right as "there is". That is what makes each step of a chain like $2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)$ safe to read on its own.
- **C** × One lucky pair proves nothing, and almost any equation would satisfy this. The left is chosen against you; the right is chosen afterwards, knowing what the left turned out to be.
- **D** × It has a precise agreed meaning, which is exactly why the rule is worth stating. Without it, T(n) = 2T(n/2) + Θ(n) would be a shorthand rather than a statement.

Takeaway. Each equals sign in a chain like this points one way: every step is allowed to lose detail and no step is allowed to invent it.

4.4

True: **A, C, D, E** · False: B, F

- **A** ✓ 2^{n+1} is $2 \cdot 2^n$, a constant factor, so it is bounded on both sides at once. An offset in the exponent is a constant factor in disguise.
- **B** × 2^{2n} is 4^n, and $4^n / 2^n$ is 2^n, which passes every constant. A factor in the exponent changes the base, and no constant survives that.
- **C** ✓ Upper: n! is a product of n factors, none above n, so $n! \le n^n$ and $\lg(n!) \le n \lg n$. Lower: the top half of the factors are each at least n/2, so $n! \ge (n/2)^{(n/2)}$ and $\lg(n!) \ge (n/2)(\lg n - 1)$. Stirling's formula gives a sharper version of each, and neither sharper version is needed to place lg(n!) in this growth class.
- **D** ✓ $n^{(1/\lg n)} = 2^{(\lg n \cdot 1/\lg n)} = 2$, at every n above 1. Not merely bounded: it is the constant 2, which is why it sits at the very bottom of any ranking by growth.
- **E** ✓ Substitute lg n for n in "every exponential with base above 1 beats every polynomial" and this falls out. Any positive power of n beats any power of lg n, however lopsided the two constants are.

- **F** × Backwards. n^k / b^n goes to zero, so $n^k = o(b^n)$ and the exponential wins. It is the same fact as $\lg^k n = o(n^\varepsilon)$, before the substitution.

Takeaway. Four of these six are the facts everything else gets compared against. The two that fail, fail differently. One moves a factor into an exponent, where constants stop being affordable. The other is simply pointed the wrong way round.

4.5

Order: **E < B < C < A < G < D < F**

1. **E · n^(1/lg n)**: Equal to 2 at every n above 1, so it is a constant wearing a complicated hat. Nothing grows more slowly than a constant.
2. **B · lg*(n)**: How many times you can take a logarithm before reaching 1. It is 5 at 2^65536 and there is no n anybody will ever write down where it is 6.
3. **C · lg lg n**: Two logarithms, which is unimaginably slow and still faster than lg*: lg* inverts a whole tower of exponentials, lg lg inverts two.
4. **A · √(lg n)**: Half a logarithm, in the sense that squaring it gives one back. It beats lg lg n because lg n outgrows $(\lg \lg n)^2$.
5. **G · $\lg^2$ n**: $(\lg n)^2$, a logarithm squared, not the logarithm of a square. Any power of a logarithm is called polylogarithmic, and every one of them is outgrown by every positive power of n, however small that power is.
6. **D · 2^√(2 lg n)**: The awkward one, and the reason the ladder is not just powers and logarithms. Its logarithm is √(2 lg n), which outgrows any multiple of lg lg n, so it passes every $\lg^k n$. But it equals n^√(2/lg n), whose exponent goes to zero, so it stays under every n^ε. It lives in the gap between polylogarithmic and polynomial.
7. **F · n lg n**: Linear with a logarithm on it, so it grows faster than every other row here, and it is the cost of sorting by comparisons.

Takeaway. The middle of this list is where intuition runs out. Between "a power of a logarithm" and "a power of n" there is a whole range of functions, and 2^√(2 lg n) is one of them.

4.6

Answer: **C**

- **A** × They line up with <, = and > and they do not cover every pair. n against n^(1 + sin n) is none of the three, because the exponent keeps moving.
- **B** × Polynomials are the easy case: two of them are always comparable, because their degrees are. The trouble starts as soon as an exponent is allowed to move.
- **C** ✓ n and n^(1 + sin n) are such a pair. The exponent swings between 0 and 2 forever, so the ratio is unbounded above and below and never settles. Nothing in the definitions requires two functions to be comparable.
- **D** × That fails on the same pair for the same reason: n^(sin n) has no limit. The limit forms of o and ω are a convenience where the limit exists, not a more general definition.

Takeaway. Any two numbers can be compared. Any two functions cannot, and that is the one place where reading these five symbols as inequalities will mislead you.

4.7

True: **B, D, F** · False: A, C, E

- **A** × $n = O(n^2)$ and n^2 is not O(n). O is a ceiling: being underneath somebody says nothing about them being underneath you.
- **B** ✓ Transpose symmetry, straight out of the definitions: the single inequality $f(n) \le c \cdot g(n)$ read from the other side is $g(n) \ge (1/c) \cdot f(n)$.
- **C** × $2n = O(n)$, and $2^{(2n)} = 4^n$ is not $O(2^n)$. Exponentiating turns a constant factor in the exponent into a change of base, and no constant pays for that.

- **D** ✓ $f \le c \cdot g$ gives $\lg f \le \lg c + \lg g$, and $\lg c$ is a constant while $\lg g$ is at least 1, so $\lg f \le (1 + \lg c) \cdot \lg g$. The two side conditions are exactly what stop $\lg g$ from being zero or negative underneath.
- **E** × Take $f(n) = 1/n$, which is positive at every n. Then f / f^2 is n, which passes every constant. Squaring only helps a function that is at least 1, and asymptotically positive does not say that.
- **F** ✓ Anything in $o(f)$ is eventually below f, so the sum sits between f and 2f from that point: $c_1 = 1$ and $c_2 = 2$. This is "lower order terms do not change the class", stated so that it can be checked.

Takeaway. Three of six. Every false one here is a plausible-sounding rule that somebody has written in an exam, and each is killed by one small counterexample rather than by a subtlety.

4.8

Any pair that really holds is correct. There is nothing to match against.

$c = 1$ and $n_0 = 1$ work, because $n! \le n^n$. The bound is also tight in the Θ sense: the largest half of the factors are each at least $n/2$, so $n! \ge (n/2)^{(n/2)}$ and $\lg(n!) \ge (n/2)(\lg n - 1)$, which is $\Omega(n \lg n)$.

- **If you gave $c = 1$, $n_0 = 1$:** The tightest pair there is, and it needs no real threshold: $n!$ is a product of n factors none of which exceeds n, so $n! \le n^n$, and taking logarithms of both sides is the whole proof. The two sides are equal at $n = 1$, where both are 0.
- **If your c was below 1:** A constant below 1 cannot work, however large n_0 is. $\lg(n!)$ is $n \lg n - n \lg e + O(\lg n)$ by Stirling, so the ratio $\lg(n!) / (n \lg n)$ climbs back towards 1 and passes any constant below it.

Takeaway. $\lg(n!) = \Theta(n \lg n)$ is the fact underneath the lower bound for comparison sorting: $n!$ possible orderings need about $\lg(n!)$ yes-or-no questions to tell apart, and that is $n \lg n$ up to a constant.

4.9

Lines: **1A · 2B · 3A · 4C**

LINE 1 · WHAT HAS TO BE PRODUCED

- **A** ✓ The definition of Θ, written out. Two constants and one threshold, and the whole proof is finding them.
- **B** × True, and it is what max means rather than something to prove. No amount of restating it produces a constant.
- **C** × That is O, which is half the job. Θ is an upper bound and a lower bound holding together, and the lower one is where the thinking is.

LINE 2 · THE UPPER BOUND

- **A** × Not in general, and not the right quantity either. With f and g both equal to 1/2 past some n, the product is smaller than either one.
- **B** ✓ Adding a nonnegative quantity to the larger of the two cannot make it smaller. Nothing about the shape of either function is used, only that neither goes below zero.
- **C** × The hypothesis is unnecessary, and adding one you do not need weakens the result for no reason. Nonnegative is the whole requirement.

LINE 3 · THE LOWER BOUND

- **A** ✓ The step that carries the proof. It is the only one that relates max to the sum rather than to one of the two functions, which is what the claim is about.
- **B** × True, and it bounds max below in terms of f rather than in terms of $f + g$. With g far larger than f it says nothing about the sum.
- **C** × A true inequality between the wrong two quantities. min tells you nothing about how large $f + g$ is.

LINE 4 · PUTTING IT TOGETHER

- **A** × Transpose symmetry turns $f = O(g)$ into $g = \Omega(f)$, which is a statement about the other function. It never upgrades a one-sided bound into a two-sided one; only having both bounds on the same pair does that.
- **B** × Equal is what Θ reads like and not what it says. Take $f = g = n$: the max is n, the sum is $2n$, and they are different functions in the same class.
- **C** ✓ Both bounds, one threshold, two named constants. Nothing asks for the constants to be the best possible ones.

Takeaway. Every Θ proof has this shape: an upper bound, a lower bound, and one threshold past which both hold. What makes this one instructive is that the lower bound is the half that needs an idea, and the idea is as small as "the larger of two numbers is at least their average".

5.1

Answer: **B**

- **A** × It is not, and nobody claims it is. The argument for worst case is that it is a guarantee you can state without knowing anything about the inputs, not that it is typical.
- **B** ✓ An average-case result is a result about the distribution somebody assumed, and the same algorithm can look fast under one choice and slow under another. Worst case needs no such assumption, which is why it survives contact with inputs nobody predicted.
- **C** × It is often harder and it is done all the time, quicksort being the standard example. The reason it is not the default is that its answer depends on an assumption the analysis has to smuggle in.
- **D** × O bounds whatever function it is pointed at, average case included. Which running time is being bounded and which bound is being used are two separate things to say, and both have to be said.

Takeaway. A worst-case bound is a promise about every input. An average-case bound is a promise about a distribution, and the distribution is the part that has to be argued for.

5.2

Answer: **A**

- **A** ✓ $c(2n)^d = 2^d \cdot cn^d$, so the slowdown is 2^d whatever n was. An exponential has no such constant: doubling the input squares 2^N. That gap, rather than any particular threshold of acceptable speed, is what the definition is drawing a line around.
- **B** × Often true and not what the definition says: n^100 is polynomial and hopeless. The definition earns its place empirically, because problems with polynomial algorithms nearly always turn out to have n, n log n, n^2 or n^3 ones, but that is a fact about the world rather than part of the definition.
- **C** × Computers run exponential algorithms every day on small inputs. The line is about how the cost responds to growth, not about what is executable.
- **D** × n^3 is a polynomial and is not $O(n^2)$. Nothing bounds the degree, and that is deliberate: a definition that fixed a degree would have to be revised every time somebody found an n^2.5 algorithm.

Takeaway. The definition is about scaling, not speed. It survives because the polynomials that turn up in practice are small ones, which is luck rather than logic, and everybody who uses the definition knows it.

5.3

Answer: **× 8**

$c(2n)^3 = c \cdot 8n^3 = 8 \cdot cn^3$. In general the factor is 2^d, which is a constant because d is, and that is the whole content of "polynomial algorithms scale well".

- **If you wrote 2:** That is the factor the input grew by. The running time follows it through the exponent: $c(2n)^3 = 8cn^3$.
- **If you wrote 3:** That is d. The factor is 2^d, not d: three doublings of the work, one for each factor of n in n^3.
- **If you wrote 6:** 2 × 3 rather than 2^3. The n in cn^d is raised to the power, not multiplied by it.

Takeaway. The number to carry away is that the slowdown factor does not depend on n. For 2^n the same question has no answer: doubling the input squares the running time, so the factor is itself exponential.

5.4

True: **A, B, C, D** · False: E, F

- **A** ✓ d does not have to be a whole number. n^1.59 is n^($\log_2 3$), the bound for multiplying two n-digit integers by splitting each in half and doing three multiplications instead of four, and it is comfortably $O(n^2)$ as well.
- **B** ✓ √n is n^0.5, so d = 0.5. Exponents below 1 are polynomial too, and this one is sublinear, meaning less work than reading the whole input. That is only possible when the input is already arranged so that most of it can be skipped, the way a sorted list lets a search ignore half of what is left at every step.
- **C** ✓ log n ≤ n for every n ≥ 1, so n log n ≤ n^2, and a bound that is $O(n^2)$ is polynomial. It is not of the form n^d itself, which is exactly why the definition says "is $O(n^d)$" rather than "is n^d".
- **D** ✓ Polynomial, and useless. The definition does not pretend otherwise: it draws its line between polynomial and exponential and says nothing about where inside the polynomials the practical boundary sits.
- **E** × The thing the definition exists to exclude. There is no constant d with $2^n = O(n^d)$, since $n^d / 2^n$ goes to zero for every d.
- **F** × The exponent contains n, so there is no constant d to name: it passes every fixed exponent eventually. It also grows slowly enough to be perfectly usable in practice, which is the honest counterexample to the definition rather than to the arithmetic.

Takeaway. Four of these six are polynomial, and one of the two that are not would be a fine algorithm to have. The line is drawn where it is because it does not shift with the hardware and is easy to check, not because it lands in exactly the right place every time.

5.5

Answer: **A**

- **A** ✓ A sum of a fixed number of functions, each O(h), is O(h): the constants add, and a sum of finitely many constants is a constant. Negative coefficients are covered because $|a_j|$ is used, and the count d + 1 has to be fixed, which it is because the degree is.
- **B** × That settles the upper bound and loses the lower one. Θ needs both, and the lower bound is where a large negative a_{d−1} does real damage for small n, which is what the threshold in the definition absorbs.
- **C** × True, and it proves the claim by a different route: a ratio tending to a positive constant gives Θ directly. It is a fine argument and it is not the one the sum rule gives, which needs no limits at all.
- **D** × Not for every n. In $p(n) = n^2 − 100n + 5000$ the constant 5000 is the largest of the three terms up to n = 50, the middle term 100n is the largest from there to n = 100, and only past n = 100 does n^2 lead. Eventually largest is what is true, and the whole point of the threshold is that eventually is enough.

Takeaway. Two rules do all the work: a fixed-size sum of things that are O(h) is O(h), and a constant multiple of something O(h) is O(h). Everything about polynomials follows from those, without ever looking at a limit.

5.6

Answer: **B**

- **A** × That is n^2, not n log n. A logarithm turns up when something halves, not when something is nested.
- **B** ✓ That recursion has solution O(n log n): the work at each level of the recursion is linear, and there are $\log_2 n$ levels because the size halves each time. Mergesort is the standard instance, and it is why sorting sets the bar at n log n.
- **C** × Sorting is the most visible instance and it is not the mechanism. It is also the second common route in its own right: an algorithm whose most expensive step is a sort, followed by a linear pass, is O(n log n) because the sort dominates.
- **D** × Also O(n log n), and a real pattern in its own right. It is n separate logarithmic operations rather than one recursive shape, and when a bound of n log n turns up without an obvious sort and without an obvious per-item logarithm, the halving recursion is what is behind it.

Takeaway. Running times come in a small number of shapes, and each shape has a usual cause: one pass gives n, halving gives log n, halving with linear work at every level gives n log n, all pairs gives n^2, all subsets gives 2^n.

5.7

Order: **H < C < F < G < E < B < A < D**

1. **H · O(log n)**: Binary search: throw away half of what is left at every step, so log n steps are enough. Less work than reading the whole input, which is possible only because a sorted list lets each step skip the half it did not look at.
2. **C · O(n)**: One pass, constant work per item. The maximum of n numbers, or a merge of two sorted lists.
3. **F · O(n log n)**: Split in half, recurse, combine in linear time. Or: sort, then make one pass over the sorted order.
4. **G · O(n^2)**: All pairs. There are $n(n-1)/2$ of them, which is at most $n^2/2$, and constant work on each. The closest pair of n points, done the obvious way.
5. **E · O(n^3)**: Three nested loops. Testing every pair of n sets for a common element, with each set of size up to n, is the standard example.
6. **B · O(n^4)**: All subsets of size k, here k = 4. There are n choose k of them, the number of ways to pick k things out of n, which is at most $n^k / k!$, and k is a constant so the factorial vanishes into the notation.
7. **A · O(2^n)**: All subsets of any size. The search space for finding a largest independent set, and for a great many problems where nobody knows how to do better.
8. **D · O(n!)**: All orderings, or all ways to pair n things with n other things. That is how many ways there are to pair n students with n schools, and it is the space a propose and turn away process gets through in n^2 steps instead of visiting.

Takeaway. The last two rows are why the definition of efficient is drawn where it is. A stable matching instance has n! perfect matchings and is solved in n^2 steps, and that gap, rather than any particular speed, is what an algorithm is for.

5.8

Answer: **A**

- **A** ✓ $\log_a n = \log_b n / \log_b a$, and $1/\log_b a$ is a constant, so $\log_a n = \Theta(\log_b n)$ and the base vanishes into c. For $r > s > 1$, $r^n = \Theta(s^n)$ would need $(r/s)^n \le c$, and $(r/s)^n$ grows without bound. So 1.5^n and 2^n are genuinely different, and at n = 50 they are 11 minutes and 36 years apart.
- **B** × One of them is a theorem. Changing the base of a logarithm multiplies it by a constant, and constants are precisely what the notation forgives.
- **C** × Big-O ignores constant factors, and the gap between r^n and s^n is not a constant factor: their ratio is itself exponential. The reason the log case is safe does not transfer.
- **D** × Being slow is not the reason. n and 2n are both fast-growing and differ by a constant too. What decides it is whether the change of base is a factor or a change of shape.

Takeaway. The question to ask is always the same: is the difference a constant factor. For logarithms it is, for exponentials it is not, and the notation forgives exactly one of those.

5.9

Answer: **A**

- **A** ✓ The input is 2n lists of n names, so its size is about $2n^2$, and the space of candidate answers is n!, which is astronomically larger. Nearly every problem worth studying has this shape, and finding the structure that lets you skip the enumeration is what algorithm design is.
- **B** × It looks exactly as hard as it is, which is the point. Interval scheduling has a 2^n search space and an O(n log n) algorithm; finding a largest independent set has the same 2^n space and no known polynomial algorithm. The size of the space predicts nothing on its own.
- **C** × Nothing is wrong with the implementation. Checking all n! matchings is n! work however carefully it is written, and at n = 20 that is more than 10^{18} of them.
- **D** × n! really is the number of candidate answers, and the count is right. What the algorithm shows is that the answer can be found without visiting them, which is a different claim from the count being wrong.

Takeaway. Efficient, in the sense that matters, means beating brute force by a change of shape rather than by a constant. That is what the whole vocabulary of this page is for: it is the only way to say "by a change of shape" precisely.